From Scratch:
The Design and Implementation of a SAT Solver
(Second Draft)

Oliver Lie

September 8, 2026

Contents

Preface

Hello, thank you for picking up this book. I have just finished writing it, except for this portion which I'm writing right now! It has been one epicly long journey of 9 weeks in my last quarter of college before I graduate. I wasn't even sure if I would be able to complete this by the end, but it seems just like my assignments, I finished this at the last minute.

To be completely honest, I am not an expert when it comes to computer science. After all, my degree is in environmental sciences (just kidding, I am a computer science major). However, even though I don't know everything about every data structure, algorithm, design pattern, or whatever, I think that this book would be great for someone getting started in the world of SAT solving. These things that I am not an expert in are my passion, and by doing this technical research, I hope to become a little less of "not an expert" in those fields.

We are not going to revolutionize SAT solving, in fact, we're barely going to scratch the surface in this book (it's a big world out there). What I aim to do in writing this is to help you understand how SAT solvers work and how they are implemented. I hope that this remains accessible and I'm not too confusing, however, I am just a ~~girl~~ undergraduate. Because we're covering how SAT solvers work and are implemented, if you are a student tasked with building one, don't use this to cheat. That would hurt my feelings. Instead use this to help your own understanding instead of copy-pasting.

Throughout this book we will build our own modern SAT solver from scratch, complete with all the bells and whistles. We will begin with the most basic algorithm imaginable, brute force, and gradually work our way through DPLL, CDCL, and beyond.

Throughout this journey we will encounter thought experiments, exercises, and a handful of experiments intended to solidify our understanding. Any external resources will be cited (hopefully correctly), and in keeping with the goal of accessibility:

- Questions that are explicitly asked will eventually be answered.
- Proofs will be explained in informal language before becoming formal.
- Code will be provided and linked through GitHub (or another hosting platform).
- Revisions to this book will be made whenever I discover mistakes or better explanations.

There is one more thing that deserves acknowledgement: some pieces of code were AI-assisted. To be completely transparent, "AI-assisted" means that instead of spending

days repeatedly banging my head against a wall trying to understand a bug or some subtle algorithmic detail, I occasionally asked an AI to explain concepts or help debug an implementation. Every word of this book, however, is my own. AI did not generate any of the explanatory text you will read here, although it certainly helped produce some of the code that accompanies it. I know that AI is very unpopular, and I do get that, however, even with literature and code I could find, it wasn't enough **for me** to figure out how everything is implemented from the ground up. That being said, I would still encourage you to read the other literature out there and look at code on your own, and perhaps you'll find me an idiot for needing AI assistance in the first place.

There are practical reasons for this. SAT solvers are difficult to implement correctly, and introducing subtle bugs is remarkably easy. In fact, before deciding to write this book, I had already attempted to build a SAT solver once before. I never managed to get a working CDCL implementation.

That failure, however, taught me a tremendous amount. So in a sense, we are learning SAT solving together (isn't that comforting?). My previous mistakes have shown me many of the pitfalls, and my hope is that by documenting this journey I can help you avoid making the same ones. So without further ado, let's begin. By the time we reach the end, I hope I will have taught you more than I myself learned.

This version of *From Scratch* is the second draft. In this version, we are going to improve our handling of pseudocode, and better explore differences between pseudocode and the actual implementation in our SAT solver. I hope that this helps you understand the material better.

Chapter 1

Introduction

SAT solvers, in my opinion, are one of the most underrated pieces of algorithmic technology of the modern day. They are used in both software and hardware verification (circuitry testing), product configuration, package management, computational biology, particle physics, solving graph problems, and much much more. [1]

In a more computer science-focused world, they have applications in NP Completeness. Since (nearly) [2] every problem has a reduction to a Boolean SAT problem, creating one really really good SAT solver lets us solve all those problems in a "fast" manner. So they are important everywhere despite the vast majority of us never really thinking about their existence. One note on what "fast" means: even if solving a SAT problem takes a small amount of time, the algorithmic complexity isn't necessarily "fast" as there is no known polynomial algorithm for any NP-Complete problem (I am a firm believer that $P = NP$, and yes I know that makes me crazy).

Before we start writing out code, proofs, and the like, we need to make sure we're all on the same page on things such as input and ideas. Reading through this introduction is important so that you understand the basic notation used throughout this book.

One last thing to note: we will write out pseudocode rather than the code for one specific programming language. In the first draft, I used a C-based syntax for readability purposes. However, what I thought was a readability issue was actually my lack of ability to properly format pseudocode using LaTeX. So in this version, we will use actual pseudocode syntax, and therefore, some proficiency with pseudocode is required to read this. Luckily, I will still explain what's going on, and I think you'll be able to pick it up quite easily. Just like in the first version, we will pretend our arrays and other data structures are dynamically resizable and don't require any pre-allocation of space. [3] That being said, you will see "our interface" sections every now and then when we make changes to our SAT solver object. The syntax in those sections will be more C++ based than pseudocode based because that is part of our actual implementation.

[1] `https://www.cs.utexas.edu/~isil/cs389L/lecture4-6up.pdf`

[2] Before anyone comes after me, the Halting Problem doesn't have a reduction to a boolean satisfiability problem, so take that.

[3] If you are implementing this in real code, this tiny detail of no pre-allocation of memory will come back to bite you later, so if you are encountering segfaults, this *should* be one of the first things you should check!

1.1 What Do SAT Solvers Even Solve?

"What do SAT solvers even solve," I hear you say? SAT solvers solve *boolean satisfiability* problems. Another way to say that is they solve *propositional logic* problems. There are many forms of propositional logic, and you may be familiar with it. Here are some basic examples:

- $P \rightarrow Q$, an implication statement
- $(A \wedge B)$, an and statement
- $(A \vee B)$, an or statement
- $\neg A$, a negation/not statement

Of course these are very basic examples, and we haven't included every operator such as xor, nand, etc. As you know, we can rewrite implications to contain only and, or, and not operators, which is what SAT solvers operate on. In other words, SAT solvers only operate on Boolean algebra equations. [4]

However, SAT solvers don't operate on just any Boolean algebra equation. No no no, they must be in a proper form, called CNF (more on that in the next section). CNF (Conjunctive Normal Form) equations are a "conjunction of disjunctions," i.e., clauses of ors, combined with ands.

Some examples of CNF are as follows:

- $(A \vee B)$
- $(A \vee B) \wedge (\neg C)$
- $(A \vee B) \wedge (\neg C \vee D)$

Some examples that are **not** CNF:

- $\neg(A \vee B)$, where there is a NOT in front of a clause
- $\neg(A \vee B) \wedge (C)$, where there is a NOT in front of a clause
- $(A \vee B \wedge C)$, where there is an AND within a clause
- $(A \wedge B) \vee (C)$, where it is in DNF form. [5]

[4] A Boolean algebra equation is one containing only and ($\wedge$), or ($\vee$), and not ($\neg$) operations (at a basic level).

[5] DNF (Disjunctive Normal Form) is a "disjunction of conjunctions," i.e., clauses of ands combined by ors. Solving these is trivially easy, as we just need to find one clause where no element evaluates to false.

1.2 Our Environment

Before we begin, we should talk about the environment that this book uses.

- Hardware: an Apple M1 MacBook Air, running MacOS Sequoia 15.7.8 with 8GB RAM and 512GB of storage space.
- Programming Language: C++20 compiled by Apple clang version 17.0.0 (clang-1700.6.4.2) targetted for arm64-apple-darwin24.6.0
- IDE: Visual Studio Code version 1.132.1
- Testing and Benchmarking: Testing and benchmarking is done by Test++, a C++ unit testing program I wrote (and is open source in case you want to try it out). [6] As of version 20.1.3, the program itself is compiled to be optimized, and you can choose to set the optimization flags of the program you're compiling. This potentially can change the runtime of our program. This is important because we will be using the amount of time Test++ takes to finish a test suite as the time our solver took to solve each problem, even though Test++ is not technically a benchmarking program. The way tests are set up is that each category of test/testing-set is given its own Test++ test which is run to completion. The resulting time to complete that Test++ test is used as the benchmark time. Differences in benchmarking frameworks and the way tests are set up may affect the result, but our testing workflow is as follows:

    1. Create a Test++ testing function
    2. Read in the testing file(s) from the given file path/directory
    3. Create an instance of our SAT solver
    4. Solve the equation
    5. Check that the result is correct
    6. Repeat steps 3-5 until no more testing files need to be processed

1.3 CNF Encoding

Now that we know what our inputs look like, the next question (I hope you're asking questions) is "how do we encode that?" Well good thing you're reading this, because that's what we're here to talk about! In this section, we'll expand our base of boolean algebra, including some definitions, transformations, and finally good encoding.

Definitions We'll Use

Clause A clause is a collection of variables, in which each variable is separated by an or ($\vee$), and each clause is separated by an and ($\wedge$).

[6] Not a shameless plug because it's awesome.

Variable A variable is an element of a clause, taking on a value of True, False, or Unassigned. Each variable can be negated ($\neg$) or not, affecting its evaluated value.

Literal A literal is an evaluated variable, evaluating to one of True, False, or Unassigned. Since a literal is an evaluated variable, it cannot be negated. A literal whose corresponding variable is unassigned evaluates to Unassigned, regardless of whether or not the variable is negated.

A quick example of the difference of a literal value, using all three of our definitions:

$$(\neg A)$$

is a clause of one variable "A", which is negated. If the variable $A = False$, then its literal value is $True$, because

$$\neg False \equiv True.$$

In other words, A was assigned the value False, and then it evaluated to True. This will be very important in the future, so please take the time to understand it well enough.

As discussed earlier, a CNF equation only has three operators: and ($\wedge$), or ($\vee$), and not ($\neg$), but in propositional logic, there are many more operators, and their clauses are not guaranteed to be in the format we want. There are many algorithms to transform any propositional logic equation into CNF, but frankly, that's beyond the scope of this book (it's also beyond my scope). Instead, we will just look at how we transform each operator into an equivalent format using just our three operators.

Implication ($\rightarrow$)

p	q	$\neg p$	$p \rightarrow q$	$\neg p \vee q$
T	T	F	T	T
T	F	F	F	F
F	T	T	T	T
F	F	T	T	T

Xor ($\oplus$)

p	q	$\neg p$	$\neg q$	$p \oplus q$	$p \vee q$	$\neg p \vee \neg q$	$(p \vee q) \wedge (\neg p \vee \neg q)$
T	T	F	F	F	T	F	F
T	F	F	T	T	T	T	T
F	T	T	F	T	T	T	T
F	F	T	T	F	F	T	F

Biconditional ($\leftrightarrow$)

p	q	$p \leftrightarrow q$	$p \rightarrow q$	$q \rightarrow p$	$(\neg p \vee q) \wedge (\neg q \vee p)$
T	T	T	T	T	T
T	F	F	F	T	F
F	T	F	T	F	F
F	F	T	T	T	T

We won't look at negations of operators such as nand, nor, xnor, or any others. I'm not a boolean algebra expert when it comes to what operators exist. But if you were given an equation with these extra operators, you could easily apply the correct equivalency, then apply an algorithm that converts any boolean algebra equation into one of CNF form.[7]

DIMACS CNF Format

Now that we know what CNF looks like, we can now discuss how it's formatted for a SAT solver's consumption. Typically, CNF is formatted in *DIMACS CNF* form.

Some rules for DIMACS CNF form are:

1. The file may begin with comment lines. The first character of each comment line must be a lower case letter "c". Comment lines typically occur in one section at the beginning of the file, but are allowed to appear throughout the file.
2. The comment lines are followed by the "problem" line. This begins with a lower case "p" followed by a space, followed by the problem type, which for CNF files is "cnf", followed by the number of variables followed by the number of clauses.
3. The remainder of the file contains lines defining the clauses, one by one.
4. A clause is defined by listing the index of each positive literal, and the negative index of each negative literal. Indices are 1-based, and for obvious reasons the index 0 is not allowed.
5. The definition of a clause may extend beyond a single line of text.
6. The definition of a clause is terminated by a final value of "0".
7. The file terminates after the last clause is defined.

There are some things about this syntax to be wary of:

1. The definition of the next clause normally begins on a new line, but may follow, on the same line, the "0" that marks the end of the previous clause.
2. In some examples of CNF files, the definition of the last clause is not terminated by a final "0".
3. In some examples of CNF files, the rule that the variables are numbered from 1 to N is not followed. The file might declare that there are 10 variables, for instance, but allow them to be numbered 2 through 11.
4. Comments can appear anywhere in the file, but the comment must be on its own line and it must start with "c". [8]

For a quick example, here is a simple way to format a DIMACS CNF equation:

[7] I honestly may be talking out of my ass here. I will do more research on this subject at a later date.

[8] `https://jix.github.io/varisat/manual/0.2.0/formats/dimacs.html`

```
p cnf 4 2
1 2 -3 0
-1 4 0
```

And one with comments:

```
c this is a comment
p cnf 4 2
c this is another comment
1 2 -3 0
-1 4 0
```

In order to increase the compatibility of our SAT solver, we will be bending some rules.

We will firstly allow missing header lines. In the case where no header is passed in, we will instead infer the number of clauses and variables. We will also be very lenient in where comments can appear, but we will enforce that variables must be between 1 and N.

1. Comments can appear anywhere in the file, but they **MUST** be on their own line and start with "c".
2. A header is not necessary, and if one is not provided, then the number of variables and clauses shall be inferred. However, if one is provided, then the number of variables and clauses **MUST** match the header.
3. All clauses, with the exception of the last clause, must end with "0", and can extend multiple lines.
4. All variables are numbered 1 through N, where N is the number of variables. That is, if you say there are 10 variables, they must be labeled $1, 2, \ldots, 10$ and not $2, 3, \ldots, 11$ or any other convention.
5. (For compatibility) Upon encountering a "%" character outside of a comment, we will stop parsing immediately.

Thus, valid input can take on many forms:

```
p cnf 4 2
1 2 -3 0
-1 4 0
```

```
4 2
1 2 -3 0
-1 4 0
```

```
p cnf 4 2
c this comment doesn't have to be at the top
1 2 -3 0
-1 4 0
```

```
p cnf 4 2
c this comment doesn't have to be at the top
1 2 -3 0
-1 4
c the first two numbers are still the number
c of variables and the number of clauses
```

```
p cnf 4 2 1 2 -3 0
-1 4 0
```

More information can be found here.[9]

1.4 Reading a CNF Equation Programmatically

Now that we know what our input looks like, we can finally discuss reading it programmatically. Firstly we must discuss how our SAT solver will be represented in code. At the moment, we just need to keep track of

- the number of variables
- the number of clauses
- each clause in our equation
- what our solution is

```
1  public interface:
2      SATSolver(string input)
3      bool solveCNF()
4
5  private interface:
6      void parse(string equation)
7      int numVars
8      int numClauses
9      int[][] clauses
10     optional<bool>[] vars
```

Our pseudocode is as follows:

[9] https://people.sc.fsu.edu/~jburkardt/data/cnf/cnf.html

Algorithm 1 SAT solver constructor

```
1: procedure CONSTRUCTOR(input)
2:     if input is a file then
3:         content ← READFILE(input)
4:         PARSE(content)
5:     else                                  ▷ We assume input is a CNF equation
6:         PARSE(input)
7:     end if
8: end procedure
```

Algorithm 2 Input parsing

```
1:  procedure PARSE(equation)
2:      if equation contains "p cnf" then
3:          A[1..n]
4:          for line in input do
5:              if line starts with "c" then
6:                  continue on to the next line
7:              else
8:                  for token in line do
9:                      APPEND(A, token)
10:                 end for
11:             end if
12:         end for
13:         numVars ← A[1]
14:         numClauses ← A[2]
15:         idx ← 2
16:         for i ← 1 up to numClauses do
17:             Clause[1..n]
18:             while A[i] ≠ 0 do APPEND(Clause, A[i])
19:                 idx ← idx + 1
20:             end whileAPPEND(Clauses, Clause)
21:         end for
22:     else
23:         numVars ← 0
24:         numClauses ← 0
25:         A[1..n]
26:         for line in input do
27:             if line starts with "c" then
28:                 continue on to next line
29:             else
30:                 for token in line do
31:                     numVars ← MAX(numVars, token)
32:                     if token = 0 then
33:                         numClauses ← numClauses + 1
34:                     end ifAPPEND(A, token)
35:                 end for
36:             end if
37:         end for
38:         idx ← 0
39:         for i ← 1 up to numClauses do
40:             while A[i] ≠ 0 do
41:                 APPEND(Clause, A[idx])
42:                 idx ← idx + 1
43:             end while
          APPEND(Clauses, Clause)
44:         end for
45:     end if
46: end procedure
```

This code will be implemented slightly differently in the example code, as I am using C++. There is also a big bug, which is that the code assumes that because "p cnf" appears in the file, there is a header. This is not always true because it does not consider whether or not the header is on the same line as a comment, so this will break it:

```
c p cnf
1 2 -3 0
-1 4 0
```

Because it assumes that there is a header when in fact there is not. I will leave fixing this bug as an exercise to both the reader and the writer, however, I will not be implementing this fix because if you're purposely trying to break the constructor, you're not trying to solve a CNF equation anyways. In case you're wondering, the fix is to first find "p cnf" then scan backwards until you reach either the start of the file, or the end of the previous line. If you don't encounter a "c" then you know that this header is actually a header. [10]

[10] What if there are multiple "p cnf" in the equation? They all better be commented is all I have to say on that matter. In my implementation I try to stop the user from shooting themselves in the foot if they try to shoot themselves in the foot, however, it's completely valid to just let the user shoot themselves in the foot and learn not to do that.

Chapter 2

Brute Force

Since the dawn of man, whenever encountered with a difficult problem, we have decided "eh, let's do it in the hardest way possible because I can't think of a better solution." The history of brute forcing things goes all the way back to our ancestors discovering fire, [1] all the way to our current solution to climate change. [2]

The idea of brute forcing something is really as simple as it gets. We have n number of literals, and therefore 2^n possible assignments, and we will check them all until we have either found a solution, or until we've run out of assignments to check. Because of how simple the idea is, it would just be better to show code and explain that instead of explaining then showing code.

[1] This is probably not true.

[2] Which is doing nothing and hoping everything sorts itself out, which is really disappointing.

Algorithm 3 Brute Force

```
 1: procedure SOLVEBRUTEFORCE
 2:     for each possible assignment[1..numVars] do
 3:         solved ← True
 4:         for Clause[1..n] in Clauses do
 5:             satisfied ← False
 6:             for i ← 1 up to n do
 7:                 v ← Clause[i]
 8:                 idx ← ABS(v)
 9:                 if v > 0 then
10:                     satisfied ← assignment[idx] ∨ satisfied
11:                 else
12:                     satisfied ← ¬assignment[idx] ∨ satisfied
13:                 end if
14:             end for
15:             solved ← satisfied ∧ solved
16:         end for
17:         if solved = True then
18:             vars ← assignment
19:             return True
20:         end if
21:     end for
22:     return False
23: end procedure
```

Let's understand what's going on. Firstly, an *assignment* in this case is an array of boolean values. Under the current assignment, we assume that it's satisfactory (innocent until proven guilty) as shown on **line 3**. Next, we iterate through each clause in our database, and we assume that the clause is not satisfied. This is because or-ing a false value with a true value will get us a satisfied clause, but or-ing a true value with all false values will keep us at true. And if we decided to "and" everything together, it would have its own issues as well.

On **line 6**, we iterate through the current clause. We grab its literals at index *i*, and then we take the absolute value of that literal. This is because the absolute values of our literals ranges from [1..*numVars*] and each assignment has exactly *numVars* variables in it. So by taking the absolute value of a literal, we get to use it as an index into our *assignment* variable.

On **line 9**, we check if the literal is greater than 0. If it is greater than 0, we just "or" its assignment's value with whether or not the clause is currently satisfied. Otherwise, we take the negation of the assignment's value and "or" that with whether or not the clause is currently satisfied. This is because negative literals means "use the opposite value of whatever I've been assigned with." Then we "and" that clause-satisfied value with whether or not the equation has been solved. We repeat this until there are no more clauses.

Then on **line 17**, we check if we've solved the equation. If we have, we store that

assignment in our SAT solver and return *true* to signify that we've found a solution. Otherwise, we move on to the next assignment. We do that until there are no more assignments or we've found a satisfactory assignment. If we reach a point where we've looked through every assignment and none of them are satisfactory, we return $false$ on **line 22**.

Since it's trivially easy to see why the code works, we will move onto something more interesting: optimizing brute force.

2.1 Optimizing Brute Force

I'll be completely honest with you, if you want to skip this section, go ahead. It's more or less just something interesting enough we can do to make the (arguably) worst algorithm[3] better.

Since we're dealing with pseudocode, we're not going to consider the hardware side of computers such as branch prediction, cache locality, or anything that the hardware uses to execute our code (other than the CPU). We will consider it from a pure Big-O perspective (excluding constant factors unless we're choosing between two algorithms with the same computational complexity).

The first thing we will add is short circuiting to our clause evaluation. After **line 15** where we finish OR-ing the "clause satisfied" boolean, we will add in a new block:

[3] I do not know of any algorithm worse than brute forcing.

Algorithm 4 Brute Force

```
 1: procedure SolveBruteForce
 2:     for each possible assignment[1..numVars] do
 3:         solved ← True
 4:         for Clause[1..n] in Clauses do
 5:             satisfied ← False
 6:             for i ← 1 up to n do
 7:                 v ← Clause[i]
 8:                 idx ← ABS(v)
 9:                 if v > 0 then
10:                     satisfied ← assignment[idx] ∨ satisfied
11:                 else
12:                     satisfied ← ¬assignment[idx] ∨ satisfied
13:                 end if
14:                 if satisfied = True then
15:                     Move onto the next clause
16:                 end if
17:             end for
18:             solved ← satisfied ∧ solved
19:         end for
20:         if solved = True then
21:             vars ← assignment
22:             return True
23:         end if
24:     end for
25:     return False
26: end procedure
```

This just says "once this clause is satisfied, go evaluate the next one immediately."

We can then add another conditional after this loop, which will move onto the next variable assignment the instant the equation is no longer satisfied:

Algorithm 5 Brute Force

```
 1: procedure SOLVEBRUTEFORCE
 2:     for each possible assignment[1..numVars] do
 3:         solved ← True
 4:         for Clause[1..n] in Clauses do
 5:             satisfied ← False
 6:             for i ← 1 up to n do
 7:                 v ← Clause[i]
 8:                 idx ← ABS(v)
 9:                 if v > 0 then
10:                     satisfied ← assignment[idx] ∨ satisfied
11:                 else
12:                     satisfied ← ¬assignment[idx] ∨ satisfied
13:                 end if
14:                 if satisfied = True then
15:                     Move onto the next clause
16:                 end if
17:             end for
18:             solved ← satisfied ∧ solved
19:             if solved = false then
20:                 Move onto the next assignment
21:             end if
22:         end for
23:         if solved = True then
24:             vars ← assignment
25:             return True
26:         end if
27:     end for
28:     return False
29: end procedure
```

With that, that's all the optimization I can think of. In case you're wondering, no this does not change the asymptotic runtime of the method, however, it's easy to see why these changes would presumably make the algorithm run faster as we're adding code that stops evaluation the moment we've figured just enough out to know if it's worth continuing evaluation and deferring extra calculation (recording the current assignment) until once we know it's satisfiable.

As an exercise to the reader, I will ask you, how can you parallelize this algorithm? And once you've figured that out, can you program it in a way where once one thread finds a satisfactory assignment, it terminates all other threads immediately instead of waiting for the others to finish?

2.1.1 Generating Assignments

How do we generate assignments? In the C++ implementation, we don't generate an assignment directly. Instead, we can recognize that a string of 1's (true) and 0's (false) is just a string of bits, which is just a datatype. We can use a large data, such as a *long* to hold 32 variable assignments. Since this is just a data type, if we had a for loop from $i \rightarrow 2^32$, or however many variables we have involved, our index variable can be used as an assignment itself! Using some clever bit shifting, we can either store the assignment into a bit array or index the bits of i directly to simulate the behavior of accessing the assignment of a variable. You may think that 32 variables, or 64, depending on your compiler and architecture is very small, and while that may be true, you must consider how inefficient of an algorithm brute force is to begin with. Theoretically we *could* make brute force numerically scale to any number of inputs if we wanted to, but the algorithm is so terrible that after 32, it would become basically unusable anyways other than a potential CPU hog.

2.2 Brute Force - Results

To see how well our current algorithm does, running some tests would be a good idea. Why run tests when we have already proven that our algorithm works "perfectly?" To see how quickly it runs of course.

To test its speed (and as we add more features, to ensure correctness is kept), we use a database of problems from here:

`https://www.cs.ubc.ca/~hoos/SATLIB/benchm.html`

We gave our current implementation 1000 SAT instances (uf20-91), each with 20 variables and 91 clauses each, all satisfiable. It took this baseline brute force algorithm **2,119,301 milliseconds**, which is roughly 35.32 minutes. This result is actually from the first time I wrote a brute force algorithm. The brute force result you can find on GitHub, it actually took **1,770,851 milliseconds**, which is about 29.51 minutes. I have no idea what I did in the rewrite that shaved off 4 minutes. I won't even lie to you reader, I am a busy undergraduate student, so I didn't even do the "best practice" for testing, which would be to quit all other processes and then benchmark it. However, while I was testing, I was doing a Canvas assignment, so I was doing lots of background work.

Normally in science you have to run an experiment at least three times for data accuracy. But you must understand that I don't want to give up an hour and a half of my life to an algorithm that we're going to be improving upon anyways.

Here are the results for all brute force algorithms, including the short-circuited and parallel versions:

```
On all 1000 equations in uf20-91:

Brute Force:               1,770,851ms (~29.51 minutes).
Short-Circuited (Basic):   49,874ms (~50 seconds).
Parallel:                  10,589ms (~10.5 seconds).
```

For the parallel version, the way it works is that it spawns in some number of worker threads and they all share a flag that alerts them if any other thread has found a solution. If a thread has, each worker will finish whatever iteration it's on and terminate. The full code can be found on the GitHub.For the parallel version, the implementation isn't perfect. I believe that we don't prevent false sharing amongst cache lines (ie we get more cache swaps between threads leading to more cache misses), which could explain why on my M1 MacBook Air, we don't get close to a 8 times speedup. Since this was for fun, I'm not going to perfect the parallelized version, that can be an exercise to the reader :)

Chapter 3

DPLL

The Davis-Putnam-Logemann-Loveland (DPLL) algorithm is a complete backtracking based search algorithm for finding satisfiable assignments for SAT problems. It is, dare I say, the foundational algorithm for modern SAT solvers, as the state-of-the-art algorithm used by basically every SAT solver builds upon the foundations that DPLL provides, as we will see later on.
It consists of three components:

- Recursive backtracking
- Unit propagation
- Pure literal elimination

How does it work?

Imagine a binary tree of variables, each with two branches representing their assignment, one for true and one for false. Suppose we don't know anything about our CNF equation. That is, from looking at it, any variable could have any value, and we don't know how each assignment helps us in finding out whether or not the equation is satisfiable. How can we figure out if we can satisfy the equation without just brute forcing it?

The answer is ~~making an ass out of you and me~~ to make assumptions.

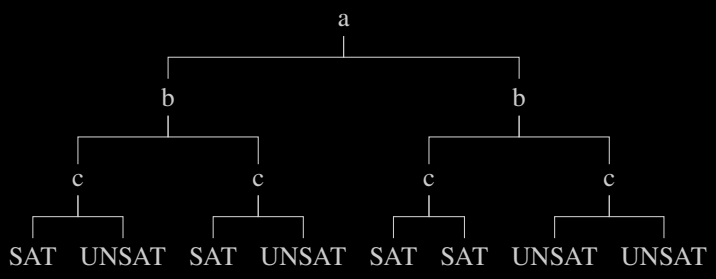


Figure 3.1: Example DPLL decision tree (the outcomes may not correspond to an actual CNF instance, sue me).

Going back to our hypothetical, we don't know anything about anything. We could have a giant decision tree to explore with an exponential number of leaves to evaluate. How does making assumptions help us? Do we just run a random number generator and if it returns 1, we go "okay, it's satisfiable," and false otherwise?

No. We make assumptions about our literal assignments.Unlike brute forcing where we try every possible combination of variable assignments at once, in DPLL we start with a literal, we'll call it A. Suppose there is nothing in the equation that indicates whether or not A should be true or false; either could be valid.

So to tackle our problem, initially we will assume A is true and see where that leads us. We propagate that value throughout our clauses, and suppose there is a clause that can't be satisfied. That's a contradiction, because if the equation was satisfiable then $A = True$ couldn't have led us there.

So we've figured out that at the very least $A = True$ leads to a contradiction. We've now eliminated half of the search space immediately and only need to consider the cases where $A = False$.

This may not make total sense immediately, so instead of giving one explanation of the algorithm and then building it, we will build up the algorithm and explain how each part works.

3.1 Recursive Backtracking

First, we need to rewrite our brute force algorithm to be recursive, as that's the foundation for the entire algorithm (duh). To do this, we will introduce a new method *solve()* which will be our recursive solving method. To also make things simpler, we will extract the equation evaluation into its own separate method as well.

This means our SAT solver becomes like so:

```
1  public interface:
2      SATSolver(string input)
3      bool solveCNF()
4
5  private interface:
6      void parse(string equation)
7      bool solve(bool[] assignment)
8      bool evaluate(bool[] assignment)
9      int numVars
10     int numClauses
11     int[][] clauses
12     optional bool[] lits
```

The flow essentially becomes like this: *solveCNF()* calls *solve()*, which calls itself over and over and returns whatever solution it finds back up to *solveCNF()*, who will handle any sort of cleanup.

Why can't we just make *solveCNF()* call itself recursively? Good question. That's because *solveCNF()* will handle any setup that *solve()* will need in order to function properly, and it will handle any cleanup that *solve()* may create.

Let's now discuss how *solve()* will work. It will be passed in a *bool[]* for the current assignment of literals. When the size of our assignment is equal to the number of variables, we will then evaluate the satisfiability of the assignment, and if it's satisfiable we return true. Otherwise, we return false, backtrack, retry, until we either find a satisfiable assignment or we've exhausted them all.

The pseudocode is as so, starting with *evaluate()*:

Algorithm 6 Evaluate CNF

```
1:  procedure EVALUATE(assignment[1..n])
2:      for each Clause[1..n] in Clauses do
3:          satisfied ← false
4:          for i ← 0 up to n do
5:              var ← Clause[i]
6:              idx ← ABS(var)
7:              if var > 0 then
8:                  satisfied ← assignment[idx] ∨ satisfied
9:              else
10:                 satisfied ← ¬assignment[idx] ∨ satisfied
11:             end if
12:             if satisfied = true then
13:                 Move onto the next clause
14:             end if
15:         end for
16:         if satisfied = false then
17:             return false
18:         end if
19:     end for
20:     vars ← assignment
21:     return true
22: end procedure
```

For this algorithm, we've extracted out the logic on how we evaluate a CNF equation, but there are a few changes to how we've done it in our brute force methods. For instance, we no longer require a global *solved* variable for the entire equation, but now we only need a *satisfied* variable for just the current clause. Since CNF is satisfied iff all clauses are satisfied, if we find one unsatisfied clause, that's enough to stop evaluation completely. Otherwise, we can save whatever assignment it was and return true.

Now for our recursive *solve()* method:

Algorithm 7 Recursive Backtracking

```
1:  procedure SOLVE(assignment[1..n])
2:      if n = numVars then
3:          return EVALUATE(assignment)
4:      end if
5:      APPEND(assignment, true)
6:      if SOLVE(assignment) then
7:          return true
8:      end if
9:      assignment[n] ← false
10:     if SOLVE(assignment) then
11:         return true
12:     end if
13:     TRUNCATE(assignment, 1)
14:     return false
15: end procedure
```

This might be slightly confusing at first, so let's go over it.

We first check our base case. If our assignment has all of its variables, we evaluate it and return the result whether it be true or false. And you'll notice that whenever we return true from a call of *solve()*, we immediately return true afterwards, so therefore, if we find a satisfiable assignment, we will go up our call stack back to *solveCNF()*, so no need to worry that it will terminate if we find a satisfiable assignment.

However, if our assignment still needs some variables, we append true to our assignment, then we do a recursive call on *solve()*. If that recursive call leads to a satisfiable assignment, we return true, otherwise, we swap out our last value for false and try *solve()* again. And if that ends up working out, we return true, but if it doesn't then we remove the last value and return false.

So how do we know that it will guarantee termination for an unsatisfiable problem? Notice how when assigning the current variable true or false and it doesn't work, we just remove the last one and go up the call stack by one? Well suppose we're at the top of the call stack (we've assigned the first variable) and the first variable is assigned false, and that didn't lead to a satisfiable assignment. Then we remove that first variable from the assignment, and then we return false. So our assignment array is empty, and we've returned to *solveCNF()*. Thus, *solve()* will terminate eventually.

And how do we know that we'll try every single assignment? Well for the very last variable, it's easy to see, if it being true doesn't satisfy the equation, we assign it false, and if that doesn't work, we remove it and go up the call stack. Then for the previous variable, if it was true, we assign it false and go back to the last variable where it tries true and/or false again. Then that doesn't work, we go up the call stack twice, and that continues until we've found a satisfiable assignment or we've exhausted them all.[1]

Lastly, our *solveCNF()* method:

[1] This is a very informal proof, but what's most important is that we build up our understanding of what's going on instead of dumping notation. Perhaps I will do a formal proof in the future.

Algorithm 8 Main Solving Method

```
1: procedure SOLVECNF
2:     assignment[1..n]
3:     return SOLVE(assignment)
4: end procedure
```

Because we're dealing with pseudocode, we don't have to do things such as reserving or resizing the assignment array. And since *evaluate()* takes care of making sure our *lits* field holds a valid assignment (if found) and holds nothing if not, we don't need to do anything else in *solveCNF()*.

You may wonder though, why not pass in *lits* to *solve()* or have no setup and make *solveCNF()* purely recursive? Those are very valid questions to ask.

This has to do with the differences between pseudocode and actual implementation. In the actual implementation, some methods depend on the state of *lits*, such as whether or not it holds anything, and by passing it into *solve()*, we force it to hold something, even if there is no satisfiable assignment. And it's true, that in *solveCNF()* if we had no setup, we could make it recursive on *lits*, however, I chose to implement it differently.

When you are reading this, please ask yourself these kinds of questions, because oftentimes, the way you read my implementation is not the best way, and even if it is, there is no reason as to why you can't think of different implementations, try them out yourself, and compare. Perhaps, and I expect this will happen often, you will find a better implementation than the one written down here.

3.1.1 Recursive Backtracking – Results

Running this code on the same 1000 instances (uf20-91), it took **1,846,644ms**, which is roughly 30.78 minutes. This is actually a pretty big improvement from our 35.32 minutes via brute forcing. Although this seems like a big gain in speed, it should be noted that at this point the algorithm is still just a fancier brute forcing algorithm. The next time I rewrote this algorithm, it took **1,540,613ms**, which is about 25.68 minutes. So once again, that's about a 5 minute speedup just from rewriting it. Perhaps we should rewrite every piece of software in the world to make it run 5 minutes faster.

Here's a question for you, why might assigning our variables true instead of false first lead to an increase in speed? **Hint: Look at the number of negations present in the testing set.**

```
On all 1000 equations in uf20-91:

Recursive Backtracking:    1,540,613ms (~25.68 minutes).
Brute Force:               1,770,851ms (~29.51 minutes).
Short-Circuited (Basic):   49,874ms (~50 seconds).
Parallel:                  10,589ms (~10.5 seconds).
```

3.2 Unit Propagation

Unit propagation is the next step in implementing DPLL, and it introduces the idea of implication. Imagine a clause with two variables: A and Z, and we know that $A = False$. What can we say about Z? Well, if our equation is satisfiable, then the clause must be satisfiable, therefore $Z = True$ because if it didn't, that would be a contradiction.

So instead of assigning variables $B, C, \ldots, Y$, then assigning Z, why don't we just do it now, and use that knowledge in other clauses containing Z or $\neg Z$?

That is essentially the idea behind unit propagation: the decisions we make imply other assignments, and so we should force them now instead of waiting until later.

However, this introduces a light hiccup into our implementation. You see, our initial invariant about assigning variables is that they are assigned in numerical order instead of (possibly) out of order.

"Not a problem! We'll just make the assignment array have space for every variable so we can index it anywhere," I hear you think. Unfortunately that interferes with our base case because it relies on the size of the assignment array. If it is initially at a fixed size of *numVars*, then our base case will always fire because our assignment array always has a size of *numVars*. So we would either need to change our base case or change how we store learned variables.

What about a learned set? One that holds whether or not we've learned a variable (positive or negative), and then as we traverse down the call stack, we just check if a variable is already learned, and if so, we give it its learned value?

That would 100% work. In fact, the first time I implemented DPLL, that is exactly what I used. However, it has the disadvantage of being annoying to maintain. Instead, why don't we create a new data type called something like *var* that represents the state of each variable: *True*, *False*, and *Unassigned*.

Then we can just maintain a simple counter of how many are not *Unassigned*, and then that keeps our base case nice and neat. Yes dear reader, we are going to do that, however, if you'd like to use the learned set, reading about how we do it without a learned set will help you and offer clues on how to implement it your way.

3.2.1 Changes To Our Implementation

```
1     public interface:
2         SATSolver(string input)
3         bool solveCNF()
4
5     private interface:
6         enum var
7         {
8             False
9             True
10            Unassigned
11        }
12
```

```
13          void parse(string equation)
14          bool solve(var[] assignment, int level)
15          bool evaluate(var[] assignment)
16          void propagate()
17
18          int numVars
19          int numClauses
20          int numAssigned
21
22          int[][] clauses
23
24          optional var[] vars
```

Here we just add in a new enum for the states that a variable can be, and adjust the methods using *bool*s to use the new enum type instead.

3.2.2 "Textbook" Unit Propagation

Before we start changing our algorithm, let's implement the *propagate()* method.

As stated before, the idea is that given a clause with either only one variable or only one *Unassigned* variable (and the rest evaluate to *False*), we can immediately say that *Unassigned* variable will be whatever value it needs to be to evaluate to *True* for that clause.

In the "textbook" way of implementation, [2] once a clause is satisfied because of a literal, you remove every other clause that is satisfied by that literal. And for every other clause containing the negation of the literal, you remove that negation entirely. This is done to shrink the search space.

We will implement this algorithm with as much malicious compliance as you can carry at an office job without getting fired:

Algorithm 9 Main Solving Method

```
1: procedure SOLVECNF(
       )
2:     assignment[1..numVars]          ▷ assignment contains numVars variables
3:     for each var in assignment do
4:         var ← Unassigned
5:     end for
       PROPAGATE(assignment)
6:     return SOLVE(assignment, 1)
7: end procedure
```

We want to propagate before we start solving in case there are clauses with only one variable (they're already unit). One thing to note about this pseudocode implementation that differs in **Algorithm 8** is that here, *assignment* already contains *numVars*

[2] https://en.wikipedia.org/wiki/Unit_propagation

variables, whereas in **Algorithm 8**, *assignment* contained up to *n* variables. They both still use 1-based indexing.

Algorithm 10 Recursive Solving Method

```
 1: procedure SOLVE(assignment[1..n], level)
 2:     if level = numVars then
 3:         return EVALUATE(assignment)
 4:     end if
 5:     backupAssignment[1..numVars] ← assignment
 6:     backupClauses[1..numClauses][1..n] ← clauses
 7:     if assignment[level] = Unassigned then
 8:         assignment[level] ← true PROPAGATE(assignment)
 9:     end if
10:     if SOLVE(assignment, level + 1) = true then
11:         return true
12:     end if
13:     clauses ← backupClauses
14:     assignment ← backupAssignment
15:     if assignment[level] = Unassigned then
16:         assignment[level] ← false PROPAGATE(assignment)
17:     end if
18:     if SOLVE(assignment, level + 1) = true then
19:         return true
20:     end if
21:     return false
22: end procedure
```

Before we get onto the code for unit propagation, let's understand this code first. Because unit propagation no longer guarantees that variables are assigned in order, we need to pass in a "level" as a parameter so we know which variable we need to assign. We also now need to check that propagation didn't already assign this variable. If it's currently Unassigned, we need to assign it then propagate, but if it's been assigned before we assign it, then we know that it's the result of propagation, so we can continue onto the next level. Other than that, it's practically the same as our recursive backtracking code, except now we need to have a "backup" of our clauses and our assignment.

Why? Because *propagate()* **mutates** the clauses and the assignment in a way that breaks the assumption that our literals are assigned in "numeric" order. Since it returns nothing, we don't have a way to undo that easily, so instead, for readability, we just have a backup copy and whenever propagation leads to an unsatisfying assignment, we return false and undo whatever changes we made by copying back into our solver's fields. And the reason why we don't need to undo the propagation before returning False is because when we move back up the call stack by one level, the method continues by undoing the effects for us. Now let's look at how we perform textbook unit propagation:

Algorithm 11 Textbook Unit Propagation

```
 1: procedure PROPAGATE(assignment[1..n])
 2:     while true do
 3:         unitClauses                                   ▷ A set of clauses
 4:         unitVars                                      ▷ A set of variables
 5:         for each Clause in Clauses do
 6:             isUnit ← true
 7:             uIdx ← −1
 8:             numU ← 0
 9:             for each var in Clause do
10:                 idx ← ABS(var)
11:                 if assignment[idx] = Unassigned then
12:                     numU ← numU + 1
13:                     uIdx ← INDEXOF(Clause, var)
14:                 else if EVALSFALSE(var) = false then
15:                     isUnit ← false
16:                 end if
17:             end for
18:             if isUnit ∧ numU = 1 then
19:                 INSERT(unitClauses, Clause)
20:                 INSERT(unitVars, uIdx)
21:                 var ← clause[uIdx]
22:                 idx ← ABS(var)
23:                 if var > 0 then
24:                     assignment[idx] ← true
25:                 else
26:                     assignment[idx] ← false
27:                 end if
28:             end if
29:         end for
30:         for each Clause in Clauses do
31:             for each var in Clause do
32:                 if CONTAINS(unitVars, −var) then
33:                     REMOVECLAUSE, -VAR()
34:                 end if
35:                 if CONTAINS(unitVars, var)∧CONTAINS(unitClause, Clause) then
36:                     REMOVECLAUSES, CLAUSE()
37:                 end if
38:             end for
39:         end for
40:         if EMPTY(unitClauses) ∧ EMPTY(unitVars) then return
41:         end if
42:     end while
43: end procedure
```

Okay, now let's understand what's happening here. We immediately go into an infinite loop (nice), and we start looking at each clause for two things:

1. The clause has only one Unassigned variable
2. Every other literal in the clause is False (remember, literals are evaluated variables)

If both things are true, then the Unassigned variable will be assigned whatever value is required to make it evaluate to True within that clause, and we know we have found a unit variable and a unit clause. After we have found all of our unit variables and unit clauses, we then rescan every other clause and delete the each unit variable's negative, (if we learn that -5 is unit, then we remove +5), and if the clause contains a unit variable, we just delete that clause entirely from the database. Once we stop finding new unit variables and new unit clauses, we stop entirely. You may wonder (I reread my code and thought I found a bug), "how do we know that we always terminate?" That's a great question. You see, this confusion may come from the fact that we rescan the database every iteration of the *while()* loop searching for unit clauses. What is stopping us from re-finding the same clause as unit upon a new iteration? The answer is much simpler than you'd expect: since we assign our unit variables, the clauses upon the next iteration no longer foloow our criteria for being unit propagate-able because there is now a literal in that clause that is True. Therefore, the number of unit variables and clauses monotonically decrease between iterations (a fancy way of saying the number can only go down).

Now, you may be wondering, "what's the performance of our solver now?" Let me answer that as plainly as possible: worse than brute force. How? Easy! We are doing SO MANY things that hurt the computer. We are allocating TONS of memory in order to backtrack, we are allocating even more memory to propagate (hash sets aren't cheap), and hashing, while O(1), is not a cheap operation to do. So yeah, I can believe that it's worse than brute force because at least the brute force algorithm was easy on the CPU. Let's learn how this is *usually* [3] implemented!

[3] I wanted to say "actually implemented", however, that's a big generalization, and again, since I'm not the expert here, I don't think I have the authority to say that. The next section is how *I* implemented it, and it roughly follows the idea on how SAT solvers implement this.

3.3 Better Unit Propagation

So the textbook style unit propagation sucked. Let's change that. As discussed before, the biggest bottleneck/inefficiency in the actual implementation is the number of copies we need to make of our clauses and assignments on each decision level. Assigning one variable creates a new decision level, so 20 variables means 20 levels. For an unsatisfiable CNF equation, I estimate we need to make about 3 copies of our clauses and assignments: one for the initial storage, another for the first failure, and a third for the second. Even if we ignore the inefficiencies in how we propagate (memory allocations, traversals, and adjustments our arrays would need to make when we delete things), this is a huge amount of memory to store. If you think that 20 variables and 91 clauses is a lot, modern SAT solvers can handle equations with **millions** of variables, so creating an algorithm that makes brute force look attractive is actually an achievement. [4]

So what's the strategy? Firstly, it would be nice if we didn't actually mutate our clauses at all. Ie, no clause deletion, and no variable deletion. Next, it would be nice if we didn't need hash sets in order to store our unit variables. Instead, it would be nice if we had a data structure where we put in our decided variables, and then it slowly shrinks as we iterate through. Ie, we propagate one on variable at a time and never again. This sounds like we need a *queue*. [5]

Lastly, instead of copying our assignment, and constantly reiterating through the *entire* clause database, what if we just marked which variables we assigned, and which clauses we've satisfied? That way when we need to undo, we just mark those variables as Unassigned, and those clauses as no longer satisfied. That way we save a lot of memory (and besides, copying is NOT O(1) as you might be led to believe by that deceptively simply "=" operator).

Let's change our interface:

```
1     public interface:
2         SATSolver(string input)
3         bool solveCNF()
4
5     private interface:
6         enum var
7         {
8             False
9             True
10            Unassigned
11        }
12
13        void parse(string equation)
14        bool solve(var[] assignment, int level)
15        bool evaluate(var[] assignment)
16        (int[] vars, int[] clauses) propagate()
17
```

[4] They might revoke my degree for that actually

[5] That wasn't shoehorned in at all!

```
18          int numVars
19          int numClauses
20          int numAssigned
21
22          int[][] clauses
23          bool[] satisfied
24
25          optional var[] vars
```

We need to make some minimal changes to *solveCNF()*:

Algorithm 12 Main Solving Method

```
1: procedure SOLVECNF
2:     assignment[1..numVars]
3:     for each var in assignment do
4:         var ← Unassigned
5:     end for
6:     PROPAGATE(assignment, 0)
7:     return SOLVE(assignment, 1)
8: end procedure
```

Since *propagate()* now takes a decision level/variable index, we need to pass one to the method. Although no variable has an index of 0 (because it's reserved to mark the end of a clause), passing in 0 just tells it "Hey, there isn't a literal to propagate on, so just find all clauses with only one variables and mark them as satisfied and 'learn' those variables." And since this is at the root level, we don't need to undo anything if it turns out that the equation is unsatisfiable, since we're gathering things that HAVE to be a certain value if this equation is to be satisfiable.

Now let's get straight to *solve()*:

Algorithm 13 Recursive Solving

```
 1: procedure SOLVE(assignment[1..n], level)
 2:     if level = numVars then
 3:         return EVALUATE(assignment)
 4:     end if
 5:     if assignment[level] != Unassigned then
 6:         SOLVE(assignment, level + 1)
 7:     end if
 8:     assignment[level] ← true
 9:     (unitVars[1..n], satClause[1..m]) ← PROPAGATE(assignment, level)
10:     if SOLVE(assignment, level + 1) = true then
11:         return true
12:     end if
13:     for each var in unitVars do
14:         idx ← ABS(var)
15:         assignment[idx] ← Unassigned
16:     end for
17:     for each cIdx in satClause do
18:         satisfied[cIdx] ← false
19:     end for
20:     assignment[level] ← false
21:     (unitVars[1..n], satClause[1..m]) ← PROPAGATE(assignment, level)
22:     if SOLVE(assignment, level + 1) = true then
23:         return true
24:     end if
25:     for each var in unitVars do
26:         idx ← ABS(var)
27:         assignment[idx] ← Unassigned
28:     end for
29:     for each cIdx in satClause do
30:         satisfied[cIdx] ← false
31:     end for
32:     assignment[level] ← Unassigned
33:     return false
34: end procedure
```

This method is practically the same, except now, we just check if we've already assigned the variable at the current decision level. If we have assigned a variable before we reach its decision level, then it must have been inferred/learned from the *propagate()* method, so we don't need to call it again. Then, it's business as usual: assume the current variable is True, propagate, if it leads to a satisfiable assignment, return True, otherwise, we undo what we've assigned and marked as learned, then repeat with False.

What's more interesting is our shiny new *propagate()* method:

Algorithm 14 Unit Propagation

```
 1: procedure PROPAGATE(assignment[1..n], level)
 2:     unit                                          ▷ a queue of unit variables
 3:     unitVars[1..n]
 4:     satClauses[1..m]
 5:     ENQUEUE(unit, level)
 6:     while Callemptyunit = false do
 7:         DEQUEUE(unit)
 8:         for i ← 1 up to numClauses do
 9:             if satisfied[i] = true then Move onto the next clause
10:             end if
11:             Clause[1..t] ← Clauses[i]
12:             uNum ← COUNTUNASSIGNED(Clause)
13:             tNum ← COUNTTRUE(Clause)
14:             if tNum > 0 then
15:                 satisfied[i] = true
16:                 APPEND(satClauses)i
17:             end if
18:             if tNum = 0 ∧ uNum = 1 then
19:                 APPEND(unitClauses, i)
20:                 uVar ← FIRSTUNASSIGNED(Clause)
21:                 APPEND(unitVars, uVar)
22:                 idx ← ABS(uVar)
23:                 if uVar > 0 then
24:                     assignment[idx] ← true
25:                 else
26:                     assignment[idx] ← false
27:                 end if
28:                 ENQUE(unit, uVar)
29:                 satisfied[i] = true
30:                 APPEND(satClauses, i)
31:             end if
32:         end for
33:     end while
34:     return (unitVars, satClauses)
35: end procedure
```

So, let's discuss what's going on. *propagate()* now takes in the decision level, which is also the same thing as an index for a variable (conceptually speaking). It then adds that to a queue, then while the queue is not empty, it scans the clauses as before. First it makes sure we're not operating on a clause that's already been satisfied (because we can't tell anything from it), and when it finds a clause that has a true literal, it marks the clause as satisfied. Then when it finds a unit clause, it marks the clause as satisfied, keeps track of the new unit variable its found, and then adds that unit variable to the queue. Then once it's done, it returns the unit variables and the indices of the satisfied

clauses.

You may have some questions about this approach. Firstly, how do we make sure we don't ever add the same variable twice? The answer is the same as before: because we assign a variable once it's found to be in a unit clause, it can't be found as Unassigned again, and therefore can't be marked as a unit variable again. Then, since we skip over already satisfied clauses, the amount of clauses we scan each iteration decreases monotonically.

And going back to our *solve()* code, even if we mark the same clause as satisfied or the same variable as unit twice, it doesn't matter because when we backtrack, we simply restore it back to its state of unsatisfied or Unassigned, respectively. What's important is that we don't mark the same clause or variable twice *across decision levels* because then, and only then, would our backtracking code become a problem because we wouldn't be correctly restoring the state of the equation back to what it was before the decision level was entered. But since we can't mark the same variable or clause twice **until** we backtrack, there's nothing to worry about.

Before we end this section, you may also be wondering: can we use the *satisfied* array in our *evaluate()* method to skip clauses we know we've already satisfied? And the answer is yes, and I would encourage you to do so in your implementations.

3.3.1 Better Unit Propagation - Results

So after implementing better unit propagation, this took **868ms** to solve all 1000 satisfiable equations. The first time I implemented this version of unit propagation, it actually took **2363ms**, so this is somehow over twice as fast. And because it's so fast, just like the first time I implemented this, we will be now testing our solver on unsatisfiable equations as well.

Our second testing set comes from the same database as before. [6] And our next testing set is called "UUF50.218.1000." In other words, it's full of equations that have 50 variables, 218 clauses, and there are 1000 such equations in the set. In order to solve all 1000 equations, it took **140,411ms** (2.34 minutes). That's also much faster than the first time I implemented this algorithm. The first time I did, it took 646,228ms (10.77 minutes). Honestly, with this much speedup, I don't know if I was just bad at implementing these algorithms the first time, [7] or if downgrading my operating system (and subsequently wiping my harddrive clean) [8] just has that much of an effect on execution speed. [9]

Why didn't we test our previous versions out on unsatisfiable equations? That's a good question. For brute forcers, since there is no satisfiable assignment, it needs to check every single one to conclude that it's unsatisfiable, which would take a lot of time. Likewise with our recursive backtracking solver. And since our (my) implementation of textbook unit propagation was *worse* than brute force, [10] obviously we wouldn't want to test it on something that will take more time than a satisfiable equation. Also because

[6] `https://www.cs.ubc.ca/~hoos/SATLIB/benchm.html`, in case you needed a reminder.

[7] Highly likely

[8] I HATED MacOS Tahoe. Sequoia is where it's at.

[9] Also likely, depending on how much RAM and swap is being used to begin with.

[10] A difficult feat.

the asymptotic runtime is *still* exponential, having a much larger equation would take a much, much larger amount of time, even if they were satisfiable.

```
Results so far:

On all 1000 equations in uf20-91:

Unit Propagation:          868ms (~0.87 seconds).
Recursive Backtracking:    1,540,613ms (~25.68 minutes).
Brute Force:               1,770,851ms (~29.51 minutes).
Short-Circuited (Basic):   49,874ms (~50 seconds).
Parallel:                  10,589ms (~10.5 seconds).

On all 1000 equations in UUF50.218.1000:

Unit Propagation: 140,041ms (~2.34 minutes).
```

3.4 Pure Literal Elimination

Suppose I gave you the following CNF equation:

```
1 2 -3 0
1 -4 5 0
1 -2 5 0
```

Do you notice anything odd about it? Perhaps “odd" is not the correct word here, do you notice an easy way to satisfy this equation? You probably noticed that the variable 1 shows up in every clause and is always in the positive. So, if you just assign $1 = T$, then it doesn't matter what the rest of the variables are assigned, because $1 = T$ satisfies every clause! This is idea behind **Pure Literal Elimination**. Essentially, every now and then, we scan our equation for “pure literals”, which are variables that only show up in the positive or negative (but never both), and then give them whatever assignment is necessary to have their literals evaluate to True.

Now, suppose I give you this equation:

```
1 0
-1 -2 -3 0
1 2 0
-2 3 0
```

I have a question for you dear reader: is -2 a pure literal? You might be inclined to say “no" because -2 and 2 are both present in the equation. However, despite that, the answer is “yes.” Why? Well notice in the first clause that 1 is a unit variable, so $1 = T$ is forced by pure literal elimination. Then the third clause, 120, because it's already satisfied, the variable 2 doesn't matter anymore. The other two clauses, which are unsatisfied, are the ones that matter. They both contain -2, and because they're unsatisfied, we can force $2 = F$ to satisfy them, thus making the assignment of 3 irrelevant.

Okay, that makes sense, sort of. How come we're allowed to do that? Like, how do we know that forcing $2 = F$ wouldn't screw up the equation elsewhere (if it were larger)? And the answer is: because it doesn't hurt the other clauses. In the third clause, once it's satisfied, the only thing that can make it unsatisfied is the backtracking, but since 1 is unit, there shouldn't be any backtracking that would suddenly make its satisfiability rest on the assignment of 2. Likewise, for the other two clauses, no matter what value 3 takes, $2 = F$ must be true to satisfy the other clause (try it out for yourself!), so we can go straight ahead and assign $2 = F$ and skip deciding.

Okay, sure, but there are 8 total assignments that we can get. How do we know that forcing $2 = F$ would lead to a satisfiable assignment? As in, how do we know we're not prematurely boxing ourselves in? That's another great question! The reason why we're not boxing ourselves in prematurely is because, again, it only helps the satisfiability of the whole equation, and because we're not doing anything detrimental, we can make that decision. [11]

[11] Essentially, think of it as making the solution “more correct” by forcing the pure literal.

3.4.1 Implementing Pure Literal Elimination

Now that we (sort of) understand why it works, how do we implement it? We need some form of efficient tracking for things we've already encountered, perhaps something like a map or a set. Then, we just need to make sure we've only encountered one version of a literal, and if we have, then we know it's pure. Then we just need to gather our pure literals and the satisfied clauses, and then return them.

Algorithm 15 Pure Literal Elimination

```
 1: procedure PUREELIM(assignment[1..n])
 2:     varToClause                                  ▷ map from variable to clause
 3:     pure[1..n]
 4:     satisfiedC[1..m]
 5:     for i ← 1 up to numClauses do
 6:         if satisfied[i] = true then
 7:             move onto the next clause
 8:         end if
 9:         Clause[1..t] ← Clauses[i]
10:         for each var in Clause do
11:             APPEND(varToClause[var], i)
12:         end for
13:     end for
14:     for each (var, clauses) in varToClause do
15:         idx ← ABS(var)
16:         if assignment[idx] ≠ Unassigned then
17:             move on to the next pairing
18:         else if CONTAINS(varToClause − var) then
19:             move on to the next pairing
20:         end if
21:         if var > 0 then
22:             assignment[idx] ← true
23:         else
24:             assignment[idx] ← false
25:         end if
26:         for each cIdx in clauses do
27:             satisfied[cIdx] = true
28:             APPEND(satisfiedC, cIdx)
29:         end for
            APPEND(pure, var)
30:     end for
31:     return (pure, satisfiedC)
32: end procedure
```

So this is a single pass version of pure literal elimination. It is possible that after finding some pure literals, and marking some clauses satisfied, a second pass could find new pure literals. So why are we implementing a single pass version? Besides it not

being a technique used in modern SAT solvers, in my experience, which isn't much since I'm being honest, it doesn't make a big (positive) difference in solve time efficiency. And besides, if we wanted a multi-pass version, we could wrap it in *while(true)* loop and break once we find no more new pure literals.

Now let's get into how the algorithm works. We first build a map from variables to their clauses (in unsatisfied clauses). Next, afterwards, we then check for each element in the map, that the map does not contain the negative variable of the current entry, and if it doesn't, we then need to make sure that the current variable is Unassigned. If the current entry passes both those conditions (pure and Unassigned), we assign it so it evaluates to True. Then, we mark each clause the variable is found in as satisfied, and then we add the indices of the satisfied clauses into the return values.

3.4.2 Pure Literal Elimination - Results

So. I have some terrible news. After implementing pure literal elimination, our solving time ***increased***. I know, I know. It's not supposed to happen, but it can happen. There are many possible explanations for this. For instance, pure literal elimination takes a decent amount of work to perform. We have to scan all unsatisfied clauses, and since there is no way to determine if a literal is impure until after everything is scanned (we could have used a hash set, but then we'd need to hash for every literal), we're accruing memory and runtime costs. Second, even if pure literal elimination finds something, it usually isn't much (at least for the testing sets currently in use), so we're doing so much work for so little in return. Third, it increases our backtracking cost because now we need to undo unit propagation and the pure literal elimination. Fourth, my implementation isn't perfect. It is possible that forcing a pure literal may force another pure literal in a different clause, so you would need multiple passes to do. However, our implementation is just one pass (wrap it in a *while* loop or something). [12]

However, no one cares *why* something performed badly. People only care about *how* badly something performed. So here are the results: For uf20.91: 1548ms, almost a 2x increase from just unit propagation. For UUF50.218.1000: 231,107ms (3.85 minutes), about a 1.5 minute increase from just unit propagation.

[12]Modern SAT solvers don't tend to use pure literal elimination for reasons outside the "it wasn't very helpful" aspect, and next chapter we'll see why.

```
On all 1000 equations in uf20-91:

Pure Literal Elimination:  1548ms (~1.55 seconds).
Unit Propagation:          868ms (~0.87 seconds).
Recursive Backtracking:    1,540,613ms (~25.68 minutes).
Brute Force:               1,770,851ms (~29.51 minutes).
Short-Circuited (Basic):   49,874ms (~50 seconds).
Parallel:                  10,589ms (~10.5 seconds).

On all 1000 equations in UUF50.218.1000:

Pure Literal Elimination: 231,107ms (~3.85 minutes).
Unit Propagation:         140,041ms (~2.34 minutes).
```

Pure Literal Elimination had about a **2x** increase (which is bad) in solving time when compared to just Unit Propagation on uf20.91, which is the test set of satisfiable problems. It also had about a **1.6x** increase in solving time on UUF50.218.1000, which is the set of unsatisfiable problems.

3.5 Watched Literals

Before we get into the Watched Literal algorithm, I want to ease us into it. Why? Because it's so different from our current version of unit propgation. For a moment (and for the rest of the book) let's forget about pure literal elimination; it wasn't that great anyways. Unit propagation gave us a massive speedup oppossed to recursive backtracking, taking runtimes down from minutes to seconds and from half-hours to mere minutes. Despite all of that, it's really inefficient in the sense that we're going so much work for so little in return.

Let's define what a **contradiction** is, because we've been using that word here and there, but beyond "unsatisfiable," we haven't formally defined what it means. A **contradiction** is the result of a literal forcing a clause to evaluate to False, leading to the whole assignment to become unsatisfiable. You might remember how in the brute force solver, we added an *if()* statement at the end of evaluating a singular clause that checked if that clause evaluated to False. If it did, we skipped that assignment and moved onto the next one, but if it didn't, we continue our evaluation.

This led to a pretty significant speedup in the brute forcer because instead of evaluating everything then deciding, we did it incrementally, allowing us to quickly leave when something was bad. At the moment, our original idea for unit propagation follows that first idea of delaying evaluation to the end. However, if you go back and look at the pseudocode, you'll notice that we do something *almost* the same as checking if a clause evaluates to False, at the moment, we check if everything except for one element evaluates to False, and that one element is Unassigned (since we made it so that unit propagation only operates on unsatisfied clauses). However, notice that when a clause evaluates to False, rather than stopping our propagation and assignment, we continue until we reach the evaluation phase?

Just like in brute force, the idea is the same: once a clause evaluates to False, no matter where we are in our assignment, there's no point in continuing on because we've found out that nothing will unfalsify that clause. As an exercise, I want you to try adding that into your unit propagation code and seeing how fast it takes to run on our test sets. I would do it, but I have a massive headache from implementing this algorithm last night. [13] Once you're done, we'll discuss watched literals some more.

So that early contradiction detection is one of the ideas behind watched literals. It's not one the key ideas, but it is part of it. There are some other ideas behind watched literals. For instance: zero cost backtracking. Not O(1) backtracking, 0 backtracking. Speaking of cost, do you notice something else wasteful about the way we propagate? If not for our *satisfied* marker array, we'd have to rescan the whole equation every time we propagated, which is about twice per level. That's pretty wasteful. After all, why do have to scan every clause in their entirety every single time? Especially because, usually, clauses don't become unsatisfied very often. And because they're not at risk of becoming unsatisfied very often, why do we need to scan it? And besides, it's not like every single variable appears in every single clause, so why not scan only the clauses we have to? I hope these questions are leading you to think of other questions regarding

[13] This headache was also induced by my lack of ability to fall asleep at all last night AND a really intense session at the gym. Computer science gives you headaches for sure, but not like this one.

inefficiencies in our current implementation.

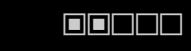
These questions were the driving force into the invention of the *Watched Literals* algorithm.

History can be found here [14]

Let's learn how the Watched Literals algorithm works. Earlier, I made a comment about how usually, each clause doesn't contain every single variable in the equation. So when we force a variable's assignment, we don't need to check every single clause, only at most the ones that contain the variable and its negation. (Ie, if we force a variable, say $2 = T$, we only need to check the clauses containing 2 and the clauses containing -2). However, do we *need* to check the clauses containing 2? Think about it. Those clauses are already satisfied by $2 = T$, so there's nothing to propagate on, for those clauses. The only clauses that have a *chance* to produce a propagation are the ones containing -2 because $-2 = F$, so if they have one Unassigned variable and everything else is now False, we can propagate!

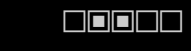
3.5.1 A Visual Example of Watched Literals

Let's view a visual example. Suppose we had a clause with 5 literals (doesn't matter what they are). The first two literals are watched



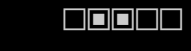
$UUUUU$

Suppose the first literal now evaluates to False. We now scan the clause for another literal that doesn't evaluate to False. We will find it at the third literal.



$FUUUU$

Now suppose the fifth literal becomes False. We do nothing because we're not watching the fifth literal.



$FUUUF$

Now the third literal becomes False. We scan again for another not-False literal.

$FUFUF$

[14] https://school.a4cp.org/summer2011/slides/Gent/SATCP3.pdf

Now the second literal becomes False. Other than the second watched literal (the fourth), there are no more not-False literals. We don't relocate the watch.

FFFUF

However, now the second watched literal knows it's the last Unassigned literal in the clause. It will now evaluate to True!

FFFTF

Our clause is now satisfied. Suppose instead, when we were here:

FFFUF

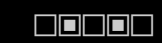
That the fourth literal also became False.

FFFFF

We've now encountered a *contradiction*, meaning that whatever assignment we have is impossible to satisfy the equation. Suppose we backtrack, and that restores us to a state where two literals in this same clause are now Unassigned. What happens to the watched literals?

UUFFF

The answer is that they stay where they are.



UUFFF

This is because one watched literal is Unassigned, so everything is okay. Although this last diagram isn't accurate to what a real restored state would look like, we can suspend our disbelief for the sake of simplicitly. The reason why watched literals work is because of the **Watched Literal Invariant**, which is that at any given point in time, at least one of the watched literals is either True or Unassigned. This invariant is what allowed us to propagate the clause when there was only one Unassigned literal left, and we will see further how it helps.

3.5.2 Algorithmic Changes

```
1  public interface:
2      SATSolver(string input)
3      bool solveCNF()
4
5  private interface:
6      enum var
7      {
8          False
9          True
10         Unassigned
11     }
12
13     void parse(string equation)
14     bool solve(var[] assignment, int level)
15     optional int[] delta propagate(var[] assignment)
16     bool createWatched()
17
18     int numVars
19     int numClauses
20
21     int[][] clauses
22     bool[] satisfied
23     queue<int> unitProp
24
25     optional var[] vars
26     map<int, int[]> var_to_clause
27     map<int, (int, int)> clause_to_vars
```

So firstly, there are some major differences. We removed the *evaluate()* method. This is because now that propagation checks for contradictions, having all variables assigned is now ***equivalent*** to finding a satisfactory assignment of variables. So we don't need to evaluate our assignment anymore! That's significantly less work than what we had before, which is what we want. Next, we added in a queue, *unitProp*, kind of like what we had for unit propagation before. This time, it's now a member of the solver object instead of a local object within the method. We will get back to that in a second. Lastly, we added in two maps, one from variables to clauses, and the other from clauses to a pair of variables.

These data structures are used for fast lookup between variables to ***indices*** of clauses (for *var_to_clause*), and one from ***indices*** of clauses to ***indices*** of watched literals. It's important that we remember we're using indices instead of actual clauses because forgetting how we're using our mapping is going to create a lot of bugs and we're gonna have a bad time.

We also added in a new method, called *createWatched()* and what it does is it creates our initial maps for var_to_clause and clause_to_vars. Here is the pseudocode:

Algorithm 16 Watched Literals Creation

```
1:  procedure CREATEWATCHED
2:      for each Clause in Clauses do
3:          if Clause has at least 2 literals then
4:              lit1 ← Clause[0]
5:              lits ← Clause[1]
6:              cIdx ← INDEXOF(Clauses, Clause)
7:              APPEND(varToClause[lit1], cIdx)
8:              APPEND(varToClause[lit2], cIdx)
9:              clauseToVar[cIdx] ← (0, 1)
10:         else if Clause is has 1 literal then
11:             lit ← Clause[0]
12:             idx ← ABS(lit)
13:             cIdx ← INDEXOF(Clauses, Clause)
14:             APPEND(varToClause[lit], cIdx)
15:             clauseToVar[cIdx] ← (0, 0)
16:             ENQUEUE(unitProp, var)
17:             if lit > 0 then
18:                 assignment[idx] ← true
19:             else
20:                 assignment[idx] ← false
21:             end if
22:         else
23:             return false
24:         end if
25:     end for
26:     return true
27: end procedure
```

So here's how to read it: for each clause, it will fall into one of three cases:

1. It's size is greater than or equal to 2
2. It only has 1 variable
3. It's empty

In the first case, the two literals that will be watching that clause will be the first two literals. So the index *i* gets appended to their array of clause indices. Then, we record that clause *i* is being watched by its first and second literal.

In the second case, since there's only one literal, it gets watched by only that literal (duh). We also record that clause *i* is being watched by only its first literal. And lastly, we enqueue that literal to our unit propagation queue so we can do a root level unit propagation before we start solving. We also want to immediately force the assignment of the variable so that way we can use its value immediately in solving.

In the last case, since there are no literals, this clause is automatically unsatisfiable, so therefore this whole equation is unsatisfiable, so we return False so that when we call this method, we know that we can't solve it.

Now let's look at our modified solving methods:

Algorithm 17 Modified Setup

```
 1: procedure SOLVECNF
 2:     assignment[1..numVars]
 3:     for each var in assignment do
 4:         var ← Unassigned
 5:     end for
 6:     if CREATEWATCHED = false then
 7:         return false
 8:     else if PROPAGATE(assignment, 0) = false then
 9:         return false
10:     end if
11:     return SOLVE(assignment, 1)
12: end procedure
```

There are some slight differences: we create our watched literals and make sure that there isn't an unsatisfiable clause, and we make sure that *propagate()* returns something. If both those things are successful, then we can start solving away.

Next, we'll modify our *solve()* method slightly:

Algorithm 18 Modified solving method

```
 1: procedure SOLVE(assignment[1..n], level)
 2:     if level = numVars then
 3:         return true
 4:     end if
 5:     if assignment[level] ≠ Unassigned then
 6:         return SOLVE(assignment, level + 1)
 7:     end if
 8:     assignment[level] ← true
 9:     pResult ← PROPAGATE(assignment, level)
10:     if propagation failed then
11:         assignment[level] ← false
12:         pResult ← PROPAGATE(assignment, −level)
13:         if propagation failed then
14:             return false
15:         end if
16:         if SOLVE(assignment, level + 1) = true then
17:             return true
18:         end if
                                          ▷ solving failed, so undo propagation changes
19:         UNDOPROPAGATION(pResult)
20:         return false
21:     end if
22:     if SOLVE(assignment, level + 1) = true then
23:         return true
24:     end if
25:     UNDOPROPAGATION(pResult)
26:     assignment[level] ← false
27:     pResult ← PROPAGATE(assignment, −level)
28:     if propagation failed then
29:         return false
30:     end if
31:     if SOLVE(assignment, level + 1) = true then
32:         return true
33:     end if
34:     UNDOPROPAGATION(pResult)
35:     assignment[level] ← Unassigned
36:     return false
37: end procedure
```

This is almost the same as our old unit propagation code, except for that we don't keep track of which clauses are satisfied across levels, and the propagation method returns an array of variable indices instead of the actual variables stored in *pResult*. Undoing propagation involves unassigning variables involved in the propagation and marking the newly satisfied clauses as unsatisfied. The code is also split up into multiple

branches depending on whether or not propagation was successful. Some parts of the code seem really similar to each other because of this. I am sure that there may be a way to simplify the similar parts, but I like the way I wrote it because I find it readable and the first time I wrote it, it was very nested.

Now for the interesting part:

Algorithm 19 Watched Literal Propagation

```
 1: procedure PROPAGATE(assignment[1..n], int decision)
 2:     delta[1..m]
 3:     if decision ≠ 0 then
 4:         ENQUEUE(unitProp, decision)
 5:         idx ← ABS(decision)
 6:         APPEND(delta, idx)
 7:     end if
 8:     while unitProp is not empty do
 9:         prop ← DEQUEUE(unitProp)
10:         if varToClause does not contain -prop then
11:             continue on to the next loop interation
12:         end if
13:         watchedClauses ← varToClause[prop]
14:         for each cIdx in watchedClauses do
15:             Clause[1..t] ← Clauses[cIdx]
16:             watches ← clauseToVar[cIdx]                ▷ pair of clause indices
17:             if Clause[watches.first] ≠ -prop then
18:                 SWAP(watches.first, watches.second)
19:             end if
20:             relocated ← false
21:             for i ← 0 up to SIZE(Clause) do
22:                 if i = watches.first ∨ i = watches.second then
23:                     continue
24:                 end if
25:                 unwatched ← Clause[i]
26:                 uIdx ← ABS(unwatched)
27:                 if EVALSFALSE(unwatched) = false then
28:                     APPEND(varToClause[unwatched], cIdx)
29:                         ▷ erase the clause index at its index within watchedClauses
30:                     wIdx ← INDEXOF(watchedClauses, cIdx)
31:                     ERASEAT(watchedClauses, wIdx)
32:                     relocated ← true
33:                     break
34:                 end if
35:             end for
36:             if relocated = false then
37:                 if CHECKVALID(Clause, watches.second, delta) = null then
38:                     return null
39:                 end if
40:             end if
41:         end for
42:     end while
43:     return delta
44: end procedure
```

Algorithm 20 check if the current assignment is valid

```
 1: procedure CHECKVALID(Clause[1..n], secondWatch, delta[1..m])
 2:     otherWatch ← Clause[secondWatch]
 3:     oIdx ← ABS(otherWatch)
 4:     if assignment[oIdx] = Unassigned then
 5:         if otherWatch > 0 then
 6:             assignment[oIdx] ← true
 7:         else
 8:             assignment[oIdx] ← false
 9:         end if
10:         APPEND(delta)oIdx
11:         ENQUEUE(unitProp)otherWatch
12:     else
13:         if EVALSFALSE(otherWatch) = false then
14:             continue onto the next loop iteration
15:         else
16:             UNDOPROPAGATION(delta)
17:             CLEAR(unitProp)
18:             return null
19:         end if
20:     end if
21: end procedure
```

This is a huge method, (so large it had to be split into two pieces so it would fit) so let's walkthrough it so we understand every aspect of it. If we look back at our *solve()* code, we pass in *cur_var* and *-cur_var* as the *decision* parameter into the propagation method. And in the beginning of *solveCNF()*, we pass in 0, to signify propagation at the root level. When that happens, we don't add anything into the queue, instead we just propagate on the literals that were forced during the creation of the watched literals.

Like I said before, we only need to check the clauses containing the negative version of the literal being propagated on because those are the clauses that the actual unit propagation could be forced upon. However, we only need to actually check the clauses being watched by the negative version of the literal being propagated on.

Next, we grab the indices of those clauses being watched and then for each one, we try to relocate the index of the first watched literal to a new literal that doesn't evaluate to False. Why only the first? Because swapping the two indices and operating on only the first index is far easier than creating a second branch dedicated to the second index. If a literal doesn't evaluate to False, we first make that unwatched literal now watch the current clause, we then make the current watched literal stop watching the current clause, and then we update the first index of the watched literals to be the index of the newly watching literal. That means we have successfully relocated the watched literal and no unit propagation needs to occur. Because we're modifying the array of clause indices, we don't want to increment the index variable in our implementation.

Aside: When you delete something in a dynamic array, in order to fill the blank space, every element to the right of what you deleted has to shift over to the left by

one unit. This takes O(n) time and on large arrays can be very costly. Because the next element has shifted over to the left, its index is now the current index, so you don't want to increment the index variable. In the implementation, the relocation logic is implemented as a "swap and pop" in which we swap the current element with the last one, then we delete the last element, which was the current element. This leads to no shifting of elements, and is logically equivalent (for our use case), even if the order of elements, relative to the deleted element, is no longer the same.

If we couldn't relocate the watched literal's index, we want to increment the index variable so that on the next iteration it will point to a new element. In the case that we couldn't relocate the watched literal, it means that every element in the clause, except for possibly the element pointed to by the second watched literal, evaluates to False. If the element pointed to by the second watched literal also evaluates to False, then we've encountered a contradiction in our assignment. However, if the second watched literal evaluates to True, then the clause is already satisfied and we need to do nothing except for move onto the next variable to propagate on. And if the second watched literal evaluates to Unassigned, then we can assign it the value it needs to evaluate to True and we then propagate on that newly assigned unit literal.

If we encounter a contradiction, we must undo all the propagation we've done, including the initial decision before returning a null value to signify that the propagation failed. However, if it has succeeded, then we return the array of the variable indices that we've assigned.

So one of the big aspects of watched literals is how we get away with doing so little amount of work. And the answer is something called the "watched literal invariant." This invariant is that at all times, at least one of the watched literals doesn't evaluate to False. I don't expect this to make everything click immediately, so let's explain more. A clause is only a contradiction when everything evaluates to False. However, we don't want to check every single clause when we assign only one variable. And even checking the negative of the variable we assigned is too much. So instead, we change which variables are watching which clauses every now and then. We pick at most two variables to watch a singular clause with, and each one watches their respective clauses until they are assigned a value that makes them evaluate to False. In that case, there is the possibility that the watched literal invariant may be violated (ie, there is a possibility that both watched literals may evaluate to False), but just to be sure, we try to change the variable watching that same clause.

We scan the clause for any unwatched literal that is Unassigned or True, and if we find one, we've restored the watched literal invariant, so we don't need to propagate or do anything else for that matter. However, if we can't find one, then we're in serious danger of the watched literal invariant being violated. So our only hope is for the second watched literal to either already be True (we need to do nothing), or Unassigned (we assign it and propagate).

Thanks to that invariant, we also don't need to do any undoing when we backtrack other than undoing what we propagated. So long as the watched literal invariant holds true when we undo our assignments, the watched literals don't need to move, which means besides the undoing of what we propagated, we get zero cost backtracking.

3.5.3 Why Two Watches?

This now begs the question, why two watched literals? What is special about the number 2? Why can't we have just one watched literal? In order to answer this question, we need to ask ourselves: "what can we do with two watched literals that we can't do with one?" When we have two watched literals, we can efficiently check for whether or not we can propagate. How so?

After we try propagating on a literal, we scan the whole clause except for the indices of the first and second watched literals. Next, if everything in the clause evaluates to False, our only hope to propagate is on the second watched literal. So if that evaluates to False as well, we have found a contradiction, and if it doesn't we have found a literal we can propagate on.

But why is that more efficient that just one watched literal? Let's think of the worst case scenario for if we just had one watched literal. What does that look like? Suppose we were watching the first literal in the clause and every other literal in the clause evaluated to False. Ie, we need to propagate on this watched literal. How would it know it needs to propagate when it was never alerted by anything becoming False? It would have no idea it needs to do anything until that watched literal was falsified, and by that point, everything in the clause would evaluate to False, so the clause would be unsatisfied, only then would we backtrack, but when we backtrack, we'd still have no way of knowing if the clause is unit.

So with only one watched literal, in the worst case scenario, which is that the literal we're watching is the last literal in the clause to be falsified, we'd be doing the same amount of work as normal unit propagation, perhaps even more.

Visually speaking, this is what I'm talking about:

UUU

The last literal becomes False, but we're not watching it so nothing moves.

UUF

Next the second literal becomes False, but we're not watching it so nothing happens.

UFF

Although we should perform unit propagation, our watched literal is completely blind to the fact that it is the last Unassigned literal in the clause, so it remains as is. If luck has it, it could become True and all would be well, but this is reality, so more likely than not, it will also become False.

FFF

Now we scan the clause, and we see that we can't relocate the watched literal, so we backtrack on our assignment.

UFF

Since the watched literal was never aware if both of the other literals became False in one pass of the algorithm or if it was across multiple decisions, it still doesn't know that it's the last Unassigned literal. It has to scan the clause each time to know whether or not it's the last Unassigned literal, which is no better than regular unit propagation.

3.5.4 Watched Literals - Results

After implementing watched literals, our runtime decreases dramatically. For uf20.91, solving all 1000 instances takes **349ms**, as opposed to the **1548ms** from pure literal elimination and the **868ms** from normal unit propagation. And for UUF50.218.1000, it takes only **7130ms** as opposed to the **231,107ms** from pure literal elimination and the **140,411ms** from normal unit propagation. Watched literals took us down from minutes to seconds for the unsatisfiable problems, which is amazing for us.

```
On all 1000 equations in uf20-91:

Watched Literals:          349ms (~0.35 seconds).
Pure Literal Elimination:  1548ms (~1.55 seconds).
Unit Propagation:          868ms (~0.87 seconds).
Recursive Backtracking:    1,540,613ms (~25.68 minutes).
Brute Force:               1,770,851ms (~29.51 minutes).
Short-Circuited (Basic):   49,874ms (~50 seconds).
Parallel:                  10,589ms (~10.5 seconds).

On all 1000 equations in UUF50.218.1000:

Watched Literals:         7,130ms (~7.13 seconds).
Pure Literal Elimination: 231,107ms (~3.85 minutes).
Unit Propagation:         140,041ms (~2.34 minutes).
```

3.6 Iterative DPLL

Now that we've implemented DPLL to the best of our ability, it's time to take it one step further. We will now implement it iteratively rather than recursively. Why? Remember how I said in the beginning that DPLL is the foundation for modern SAT solvers? [15] Well iterative DPLL is the foundation for the next chapter's algorithm: CDCL. Implementing iterative DPLL is imperative in order to create a modern SAT solver worth its salt. [16]

As said in the beginning of the chapter, DPLL forces us to traverse the decision tree as well, a tree. And that is still true, even in iterative DPLL. What changes now is that instead of an implicit path, we now have an explicit one. Suppose we are on the first decision level, so we decide variable $1 = T$, and that forces $3 = F$, $5 = F$, and $6 = T$. In recursive DPLL, we assign these variables into the *assignment* or *vars* field, but we don't really use them until we get to the decision level that matches their index or the evaluation phase.

With iterative DPLL, there are still decision levels, but the representation is now very, very different bcause instead of a decision *tree*, we now use a decision *trail*. The trail is reminiscent of a queue in the sense that we only operate at the end of the data structure. It contains some information that we need in order to continue mimicking the behavior of recursive DPLL. Each trail entry contains the literal that was forced, the decision level it's a part of, and the clause that forced that decision. We append these trail entries every single time we force a variable's assignment. The next question is how do we know which variables were decided for and which variables were inferred from propagation? Well, for decisions, there is no reason clause because there was no reason to begin with other than "because I said so." Therefore, the variables that were decided for don't get a reason clause, but the ones that were inferred do. This is great because it makes it easy to tell where the beginning of one decision starts and where it ends.

However, it would be nice to have some way to keep track of where the decisions are in the trail other than scanning for entries without a reason clause. Because of that, we will also keep another trail specifically for the decision markers. This is also helpful because then we can store whether or not we've "tried the other branch" when we backtrack.

While on the topic of backtracking, it would be nice to discuss it some more as well. Since we can't simply go up the call stack and undo variable assignments, we have to do it in the iterative way. When we backtrack, we first backtrack up to the decision entry. This is done by popping off the trail and undoing the propagated assignments. Once that's done, you then check if you've "tried the other branch", and if you haven't, you flip the decision literal's assignment and then retry. If you have already tried the other branch, you pop off the decision entry, and then repeat this process to the next decision entry. This process continues until you either find a decision that has not been switched yet, or you run out of decision entries.

With most of the exposition out of the way, I think it would be good to look at the

[15] I can't remember if I actually said that

[16] Honestly, "Salt" is a better name than "Saturn." I might change it.

changes we will make to our interface and algorithms, starting with our SAT solver.

```
1   public interface:
2       SATSolver(string input)
3       bool solveCNF()
4
5   private interface:
6       enum var
7       {
8           False
9           True
10          Unassigned
11      }
12
13      TrailEntry {
14          int lit,
15          int clauseIdx
16      }
17
18      DecisionLevel {
19          int idx,
20          bool triedOtherBranch
21      }
22
23      void parse(string equation)
24      bool propagate()
25      bool createWatched()
26
27      int numVars
28      int numClauses
29      int trailHead
30
31      int[][] clauses
32      bool[] satisfied
33
34      TrailEntry[] trail
35      DecisionLevel[] trailStarts
36
37      optional var[] vars
38      map<int, int[]> var_to_clause
39      map<int, (int, int)> clause_to_vars
```

The first notable change is that *propagate()* now returns a boolean instead of a delta. This is because it will no longer be responsible for undoing propagation in response to a contradiction, thus achieving zero-cost backtracking. Next, we now have a *trail* and a *trailHead*, which points to the last made decision entry. *trailStarts* keeps track of all the indices of the decision entries. One last thing to note is that the *TrailEntry* data structure uses indices for clauses instead of the actual clause itself. There's a very good reason for this, and I will reveal the answer in the next chapter, but until then, why do

you think it might be that way? Hint: it's not because indices are easy for accessing.

The next change we need to make is to our algorithm to create watched literals. It's the same for basically 90% of the algorithm, except for the unit clause case, which I'll highlight.

Algorithm 21 Modified createWatched

```
 1: procedure CREATEWATCHED
 2:     for each Clause in Clauses do
 3:         if Clause has at least 2 literals then
 4:             ...
 5:         else if Clause is has 1 literal then
 6:             lit ← Clause[0]
 7:             ...
 8:             cIdx ← INDEXOF(Clauses, Clause)
 9:             APPEND(trail, (lit, cIdx))
10:             ...
11:         else
12:             return false
13:         end if
14:     end for
15:     return true
16: end procedure
```

Rather than enqueuing the variable, we create a new trail entry for each unit clause. These ones are all part of the **root** decision level, which we mentally reserve for the 0th decision level. If we backtrack up to this level (if it exists) or before it, we've encountered a root level contradiction, so the equation is unsatisfiable. Because not every equation has unit clauses, it's important to remember that this decision level may not always exist, and so we should be keep that in mind for our solving algorithm.

The next algorithm we're going to change is the *propagate()* algorithm because the changes are also very minimal.

Algorithm 22 Modified propagate

```
 1: procedure PROPAGATE(assignment[1..n])
 2:     while trailHead < SIZE(trail) do
 3:         prop ← trail[trailHead].lit
 4:         trailHead ← trailHead + 1
 5:         ...
 6:         watchedClauses ← varToClause[prop]
 7:         for each cIdx in watchedClauses do
 8:             Clause[1..t] ← Clauses[cIdx]
 9:             watches ← clauseToVar[cIdx]                ▷ pair of clause indices
10:             ...
11:             if relocated = False then
12:                 ...
13:                 otherWatch ← Clause[secondWatch]
14:                 oIdx ← ABS(otherWatch)
15:                 if assignment[oIdx] = Unassigned then
16:                     ...
17:                     APPEND(trail, (otherWatch, cIdx))
18:                 else
19:                     if EVALSFALSE(otherWatch) = false then
20:                         continue onto the next loop iteration
21:                     else
22:                         UNDOPROPAGATION(delta)
23:                         CLEAR(unitProp)
24:                         return false
25:                     end if
26:                 end if
27:             end if
28:         end for
29:     end while
30:     return true
31: end procedure
```

The main difference is that we just *trailHead* to iterate through the trail, and instead of enqueuing, we just append to the trail. We also return True/False depending on whether or not propagation succeeded.

Lastly, we look at our *solve()* method, which has the most notable changes. There are two ways to implement this, depending on what makes sense to you. I call them "assignment before propagation" and "propagation before assignment." I call them these because that's what they are. Before showing the pseudocode, I want to talk about the algorithm a bit more. The beginning is mostly the same: create our watched literals, check that it succeeded, return False if it didn't. Otherwise, we do a root level propagation and then check if it succeeds as well. However, the main confusion comes from handling the case of when there is a 0th decision level and when there isn't. There are multiple ways to go about this. After the beginning, we enter the solving loop.

There are two main parts: assigning and backtracking, with a call to the propagate method separating them. Assigning is now scanning for the first Unassigned variable, forcing it True, creating a new trail entry, and then updating the trail head. I'm sure there's a better way to keep track of the next unassigned variable other than scanning the entire assignment list on each loop iteration, however, for simplicity's sake, we're implementing it the naive way.

Since we've already discussed backtracking, we will now get into the algorithm itself:

Algorithm 23 Modified solving loop

```
 1: procedure SOLVE
 2:     setupFailed ← SETUP
 3:     if setupFailed = true then return false
 4:     end if
 5:     while true do
 6:         succeeded ← PROPAGATE(assignment)
 7:         if succeeded = false then
 8:             BACKTRACK
 9:             if SIZE(trailStarts) = 0 then
10:                 return false
11:             end if
12:             continue
13:         end if
14:         if ASSIGN(assignment) = true then
15:             return true
16:         end if
17:     end while
18: end procedure
```

Algorithm 24 Setup Solve

```
1:  procedure SETUP
2:      assignment[1..numVars]
3:      for each var in assignment do
4:          var ← Unassigned
5:      end for
6:      if CREATEWATCHED(assignment) = false then
7:          return true
8:      end if
9:      APPEND(trailStarts, (0, true))
10:     if SIZE(trail) > 0 then
11:         if PROPAGATE(assignment, 0) = false then
12:             return true
13:         end if
14:     end if
15:     return false
16: end procedure
```

Algorithm 25 Backtracking code

```
1:  procedure BACKTRACK
2:      while SIZE(trailStarts) > 0 do
3:          decision ← most recent decision level
4:          while SIZE(trail) > decision.idx + 1 do
5:              lit ← last trail entry's literal
6:              assignment[ABS(lit)] ← Unassigned
7:              erase last trail entry
8:          end while
9:          if decision.TriedFalse = false then
10:             entry ← last trail entry
11:             entry.lit ← -entry.lit
12:             assignment[ABS(lit)] ← false
13:             decision.TriedFalse ← true
14:             trailHead ← decision.idx
15:             break
16:         else
17:             lit ← last trail entry's lit
18:             assignment[ABS(lit)] ← Unassigned
19:             erase last trail entry
20:             erase last trailStarts entry
21:         end if
22:     end while
23: end procedure
```

Algorithm 26 Assignment code

```
1:  procedure ASSIGN
2:      firstUnassigned ← -1
3:      for i ← 1 up to numVars do
4:          if assignment[i] = Unassigned then
5:              firstUnassigned ← i
6:              break
7:          end if
8:      end for
9:      if firstUnassigned = -1 then
10:         return true
11:     end if
12:     assignment[firstUnassigned] = true
13:     APPEND(trail, (firstUnassigned, -1))
14:     APPEND(trailStarts, (trail size - 1, false))
15:     trailHead ← trailStarts.last.idx
16:     return false
17: end procedure
```

Here is the pseudocode for solving, written as "propagate then assign." Quick question: when there's no 0th decision level and we propagate, how do we know that none of the propagated variables will have the wrong decision level? Ie, how do we know that they'll have a decision level of 0 instead of 1, since we haven't made any decisions yet? This is kind of a trick question, the answer is that if there is no 0th level, *propagate()* will do nothing because the while-loop condition will immediately return False, and so the method will terminate, assigning nothing!

Instead of showing you the code for "assign then propagate" (it is NOT just taking the assigning code and moving it up before the call to *propagate()*), let's run through how this works. When there's a 0th level, we obviously want to mark it in our *trailStart*, but why do we want to mark that the "other branch" has already been tried? Well that's because those assigned by the root level propagation are forced into those values, so it doesn't make sense to swap their values to be their opposites since it will immediately be an unsatisfiable assignment anyways.

Next, in the main solving loop, the backtracking code is just "grab the last decision index, undo until the root decision, if it's already tried False, undo the decision entirely and repeat with the previous one, and if not, try False and continue."

Lastly, let's talk about doing assign then propagation. Here's why it doesn't make total sense to start out with propagation:

- we do a root level propagation if there needs to be one.
- if there is no root level propagation to be done, it still doesn't make sense to immediately propagate since there's nothing to propagate.
- if there was a root level propagation, it doesn't make sense to do another one immediately since we've just completed one.

So if you wanted to do assign and then propagate, how would that be done? Earlier, I said that it's not just taking the code for assigning and then moving it before propagation, but why? Well, suppose we fail a propagation. We backtrack, and then suppose we haven't tried the other branch for that decision. We encounter a *continue* statement and we go back to the top. If we simply move the assigning code up to the top of the loop, what's the issue? Well remember, we just flipped a decision, so we should actually be ***propagating*** instead of assigning. If we assign immediately after flipping the decision, we now have two decisions at the same time, and that's bad because what if they are contradictory to one another, ie one would force a variable False and the other True? Unfortunately, the propagation code wouldn't catch that because it assumes everything in the trail is valid, but we just made the trail invalid. So to the propagation code, instead of it catching that a variable is forced to be False, but it's True, or vice versa, it may just ignore it because it focuses mainly on Unassigned and False literals.

So if we want to do assign then propagate, how do we make sure that we don't accidentally push something onto the trail when we shouldn't have? We need to wrap the assigning code in a conditional statement that checks that *trailHead == trail.size*, if it doesn't, then that means that there's something to be propagated on because we're not at the end of the trail, but if it does, then we've reached the end, so we should check if we've assigned everything and can be done, or if we need to assign something new.

3.6.1 Iterative DPLL - Results

Here are the results after this huge refactor:

(Assignment before propagation)

uf20.91: **328ms**, down 21ms from **349ms** by Watched Literals

UUF50.218.1000: **6369ms**, down 761ms from **7130ms** by Watched Literals

(Propagation before assignment)

uf20.91: **324ms**

UUF50.218.100: **6365ms**

```
On all 1000 equations in uf20-91:

Iterative DPLL (AbP):      328ms (~0.33 seconds).
Iterative DPLL (PbA):      324ms (~0.32 seconds).
Watched Literals:          349ms (~0.35 seconds).
Pure Literal Elimination:  1548ms (~1.55 seconds).
Unit Propagation:          868ms (~0.87 seconds).
Recursive Backtracking:    1,540,613ms (~25.68 minutes).
Brute Force:               1,770,851ms (~29.51 minutes).
Short-Circuited (Basic):   49,874ms (~50 seconds).
Parallel:                  10,589ms (~10.5 seconds).

On all 1000 equations in UUF50.218.1000:

Iterative DPLL (AbP):     6,369ms (~6.37 seconds).
Iterative DPLL (PbA):     6,365ms (~6.37 seconds).
Watched Literals:         7,130ms (~7.13 seconds).
Pure Literal Elimination: 231,107ms (~3.85 minutes).
Unit Propagation:         140,041ms (~2.34 minutes).
```

Should you do propagation before assignment simply because it's faster? It really doesn't matter, you do you. However, for the rest of this book, we will be using assignment before propagation because it's easier for me to understand, and it follows online pseudocode examples better than propagate before assignment.

The speed gain on satisfiable problems is very miniscule, however, if you ever want proof that iterative algorithms are faster than recursive ones, the unsatisfiable problem time decrease is pretty good evidence.

Chapter 4

CDCL

Conflict Driven Clause Learning (CDCL), is *the* modern SAT solving algorithm. It builds off of the ideas of DPLL and takes it to a whole new level. The "conflict driven" part of CDCL means that we analyze conflicts (contradictions) and see what we can learn from them. The "clause learning" part is exactly what it sounds like, we add new clauses to our database. In other words, CDCL is about analyzing why conflicts occur when we solve and using those conflicts to learn new clauses that will help enforce new rules as to what values variables can take, essentially changing the CNF equation into one that's logically equivalent, but easier to solve. I won't lie to you, I was not able to implement this algorithm the first time I tried. I hope and pray that this time I will be successful, so that *we* will be successful together dear reader.

Aside: why have we been opting for indices so often? You see, although we live in the land of pseudocode or machine-agnosticism, implementations don't (duh). So in our previous chapter in DPLL, there were times where we used clause indices instead of pointers to clauses (obviously using copies would be terrible for performance and memory), but why did we use indices? Now that we're implementing CDCL, we're going to be adding clauses to the database quite often, and that can trigger a resize on the database. If we used pointers to clauses, after a resize triggers, the memory address of each clause would change, thus making the pointers invalid. We would need to add new code after the insertion of each clause to make sure a resize didn't trigger, and if one did, we'd need to update each pointer for each reason clause and watched literal, which would be very difficult because after a resize, we now have no way to tell which clauses mapped to which reasons and watched literals, because, well, the old mapping isn't valid and contains no information about the new addresses for everything.
[1]

[1] More information about CDCL can be found here: `https://www.cis.upenn.edu/~cis1890/files/Lecture4.pdf`

4.1 Clause Learning

The first step in creating a CDCL solver is to learn and use new clauses. There are many versions involving different ways of backtracking, variable assigning, and even how we learn new clauses, but for now, we will keep our implementation as close to our iterative DPLL version and work our way up to the more advanced and widely used versions of the algorithm.

In order to learn a new clause, we first need to understand the learning process. The big idea is that we will encounter conflicts and contradictions while assigning and propagating new literals, and we want to learn from those conflicts and contradictions in a way that would ideally prevent us from making the same mistake again. Our unit propagation method is the perfect tool for this because it already analyzes our clauses for us and can detect when a contradiction has occurred. Rather than returning True or False to indicate a successful or unsuccessful propagation respectively, when we encounter a contradiction, we want to return the clause that is unsatisfied. You may know that when we propagate, there could be more than one clause that is unsatisfied, but just not yet evaluated to be unsatisfied. Even though there could be many conflicting clauses at once, we just want to return the first one that we find. You may wonder why not learn from all of them, and the answer is that by resolving the bad decision/propagation we made, we will (hopefully) prevent ourselves from doing more propagations that lead to those extra unsatisfied clauses.

After we return the conflicting clause to the main solving method, we then want to perform "clause resolution." As stated in the beginning, there are many versions of clause resolution, mainly differing in when they/how they stop resolving. For our basic CDCL implementation, we will resolve the conflicting clause until it contains no more propagated literals, ie, it's a clause of only decisions. The reason why this is a good stopping point is because we made all decisions, and because of that, clause learning will essentially allow us to learn which decisions we can't make together.

Lastly, after we resolve on the learned clause, we want to add it to the database, and we want to initialize its watched literals so that we can use it as if it were part of our initial CNF equation. Let's look at the high level pseudocode we'll need to implement:

Algorithm 27 High Level CDCL

```
1:  procedure CDCL
2:      SETUP
3:      while Unsat = false do
4:          ASSIGN
5:          if All variables assigned then
6:              return true
7:          end if
8:          PROPAGATE
9:          if contradiction then
10:             RESOLVECONFLICT
11:             level ← CALCULATEBACKJUMP
12:             if level < 0 then
13:                 return false
14:             end if
15:             BACKJUMPTO(level)
16:             continue
17:         end if
18:     end while
19: end procedure
```

In this chapter, we will transform our DPLL code into CDCL code. The first step is to setup our interface and change some of our algorithms to be suited for CDCL. You might be curious as to how clause resolution worst. Before we discuss how clause resolution works at a theory level, let's first discuss a few implementation changes we'll need to make.

```
1       public interface:
2           SATSolver(string input)
3           bool solveCNF()
4
5       private interface:
6           enum var
7           {
8               False
9               True
10              Unassigned
11          }
12
13          TrailEntry {
14              int lit,
15              int decisionLevel,
16              int clauseIdx
17          }
18
19          DecisionLevel {
20              int idx,
```

```
21              bool triedOtherBranch
22          }
23
24          void parse(string equation)
25          bool propagate(int[] decisionLevel, int decisionLevel)
26          bool createWatched()
27          bool createWatched(int idx, int decisionLevel)
28
29          int numVars
30          int numClauses
31          int trailHead
32
33          int[][] clauses
34
35          TrailEntry[] trail
36          DecisionLevel[] trailStarts
37          int[] var_to_trail
38
39          optional var[] vars
40          map<int, int[]> var_to_clause
41          map<int, (int, int)> clause_to_vars
```

The *var_to_trail* field is the only change we'll need to make, and it is just a mapping from variable index to trail index. If you enforce the invariant that each variable maps to exactly one place in the trail and no two variables map to the same place in the trail at the same time, you don't need to initialize these values to anything.

The next thing we'll look at is our *propagate()* code since it needs very minimal changes. The change we're making now is that instead of returning a boolean value to indicate whether or not propagation was successful, we're now going to return a clause if we encounter a contradiction and nothing otherwise.

Algorithm 28 Updated Propagate

```
1:  procedure PROPAGATE(assignment[1..n], decisionLevel)
2:      while trailHead < trail length do
3:          prop ← trail[trailHead].lit
4:          trailHead ← trailHead + 1
5:          ...
6:          for i ← 0 up to watchedClause size do
7:              ...
8:              if relocated = false then
9:                  if other watched literal evaluates to false then return Clause
10:                 else
11:                     APPEND(trail, (otherWatch, decisionLevel, Clauseindex))
12:                 end if
13:             end if
14:         end for
15:     end while
16: end procedure
```

And before we look at changes to our solving method, we're going to create a new overload for *createWatched()* that takes in the index of a clause. Its only purpose is to be called on newly learned clauses, which don't have the same guarantees as the other overload. You see, the learned clause should hopefully NEVER be unsatisfied when this method operates over it. It should ALWAYS return True, so if it ever doesn't, it's a good debugging tool. Here's how it works: we traverse over the clause of the passed in index, counting the number of True, False, and Unassigned literals it has. We will then search for two indices that could be watched literals. Since the clause should never be unsatisfied, we are guaranteed to always find one (if our implementation is correct). If we can't find another and our clause has more than one literal, we will initialize the second watched literal to any arbitrary place in the clause (with the caveat that it can't be the same index as our first watched literal). We will then check if the clause is unit, and if it is, we will add it to our trail, assign the Unassigned variable, etc. Otherwise, we do nothing.

Algorithm 29 createWatched overload

```
1:  procedure CREATEWATCHED(index, decisionLevel)
2:      Clause ← Clauses[index]
3:      if clause is empty then
4:          return false
5:      end if
6:      foundFirst ← false
7:      foundSecion ← false
8:      firstIdx ← -1
9:      secondIdx ← -1
10:     numU ← 0
11:     numT ← 0
12:     for each var in Clause do
13:         if var is Unassigned then
14:             numU ← numU + 1
15:             if foundFirst = false then
16:                 foundFirst ← true
17:                 firstIdx ← index of var
18:             else
19:                 foundSecond ← true
20:                 secondIdx ← index of var
21:             end if
22:         else if var evals to true then
23:             numT ← numT + 1
24:             if foundFirst = false then
25:                 foundFirst ← true
26:                 firstIdx ← index of var
27:             else
28:                 foundSecond ← true
29:                 secondIdx ← index of var
30:             end if
31:         end if
32:     end for
33:     if clause has at least two literals then
34:         if foundSecond = false then
35:             secondIdx ← mod(firstIdx + 1, clause size)
36:         end if
37:         ADDWATCHEDLITERALSTOSOLVER(firstIdx, secondIdx)
38:     else
39:         ADDWATCHEDLITERALSTOSOLVER(firstIdx, firstIdx)
40:     end if
41:     if numT = 0 ∧ numU = 1 then
42:         APPEND(trail, (clause[firstIdx], decisionLevel, index))
43:         ASSIGNVAR(clause[firstIdx])
44:     end if
45:     return true
46: end procedure
```

Because we don't know anything about this clause, we need to gather as much information as we can right away before we decide how watched literals are spread out on the clause. When the clause has more than one literal, we use "secondIdx = (firstIdx + 1) % clause.size" because that prevents us from adding a watched literal past the end of the clause, if *firstIdx* is already on the edge.

The last method we need to discuss is our *solve* loop. Clause resolution happens after backtracking because it relies on the current trail to understand [2] why a conflict occurred. If we were to undo propagations, then the resolution operator would have no reference as to what "most recently" means, and therefore, clause resolution wouldn't work.

Algorithm 30 Initial CDCL

```
 1: procedure SOLVE
 2:     setupFailed ← SETUP
 3:     if setupFailed = true then return false
 4:     end if
 5:     decisionLevel ← 0
 6:     while true do
 7:         if ASSIGN(assignment) = true then
 8:             return true
 9:         else
10:             decisionLevel ← decisionLevel + 1
11:         end if
12:         succeeded ← PROPAGATE(assignment)
13:         if succeeded is not null then
14:             conflict ← value of succeeded
15:             learned ← RESOLVECONFLICT(conflict)
16:             if learned is empty then
17:                 return false
18:             end if
19:             BACKTRACK
20:             if SIZE(trailStarts) = 0 then
21:                 return false
22:             end if
23:             decisionLevel ← decisionLevel - 1
24:             trailHead ← trailStarts.last
25:             continue
26:         end if
27:     end while
28: end procedure
```

[2] It follows a mathematical process to transform the conflict clause into one that prevents that same conflict from ever occurring again. The algorithm does not "understand" anything. Do not anthropomorphize computer algorithms, especially artificial intelligence.

Algorithm 31 Resolve Conflict

```
1:  procedure RESOLVECONFLICT(conflict[1..n])
2:      while true do
3:          pIdx ← -1
4:          for i ← 1 up to n do
5:              var ← conflict[i]
6:              idx ← ABS(var)
7:              tIdx ← varToTrail[idx]
8:              if trail[tIdx].reasonIdx exists then
9:                  pIdx ← MAX(pIdx, t_idx)
10:             end if
11:         end for
12:         if pIdx = -1 then
13:             return conflict
14:         end if
15:         lit ← trail[idx].lit
16:         reason ← clauses[trail[idx].reasonIdx]
17:         APPEND(conflict, reason)
18:         remove all occurrences of lit and −lit from conflict
19:         remove all duplicate literals from conflict
20:     end while
21: end procedure
```

We will need to substitute in *resolveConflict()* into the main solving algorithm. It should be noted that *resolveConflict()* mutates the original parameter and then returns the mutated parameter as the learned clause. One more thing to note is that in *solve()*, some of the algorithms called are the same from our Iterative DPLL code. For that reason, I didn't rewrite them and they can be referenced and substituted as needed.

I want to point out that this isn't necessarily CDCL yet. This is more like DPLL, but with adding clauses to our database. The reason as to why this is, is because whenever we recover from a contradiction, we only go back one level. You see, CDCL does away with the idea of a "tried other branch" flag and relies on the learned clauses to guide what variable assignments are possible and which ones are contradictory. However, because we only backtrack one level, there is nothing to actually force new variable assignments in cases where the learned clause isn't a unit clause. It can happen, but it doesn't always happen, so we still need to try the other assignment since for the most part, the learned clause isn't used.

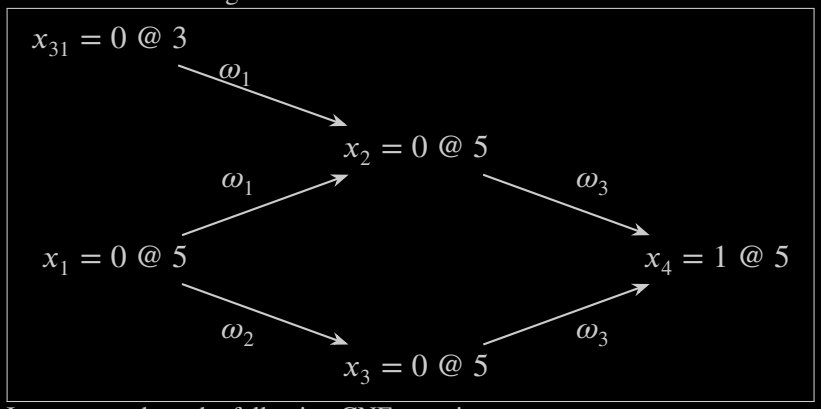
4.1.1 Clause Resolution

Although we've already seen the pseudocode for clause resolution, it's not discussed very much, and I want us to understand how it works better. As we make decisions and propagations, we form this thing called an **implication graph**. In a DPLL style solver, our assignments and propagations form a directed acyclic graph (I will not be proving this), and by traversing the implication graph, we can find out what decisions led to the contradiction.

To learn more about implication graphs, let's look at one and discuss what it means. We will use the implication graphs from here (Figures 4.1 and 4.2):

`https://www3.cs.stonybrook.edu/~cram/cse505/Fall20/Resources/cdcl.pdf`

We will start with Figure 4.1:



It corresponds to the following CNF equation:

$$(\omega_1) \wedge (\omega_2) \wedge (\omega_3) = (x_1 \vee x_{31} \vee \neg x_2) \wedge (x_1 \vee \neg x_3) \wedge (x_2 \vee x_3 \vee x_4)$$

Each node of the graph follows this form: $x_i = v@d$, but what does it mean?

This is the notation for assignment of a variable at a certain decision level:

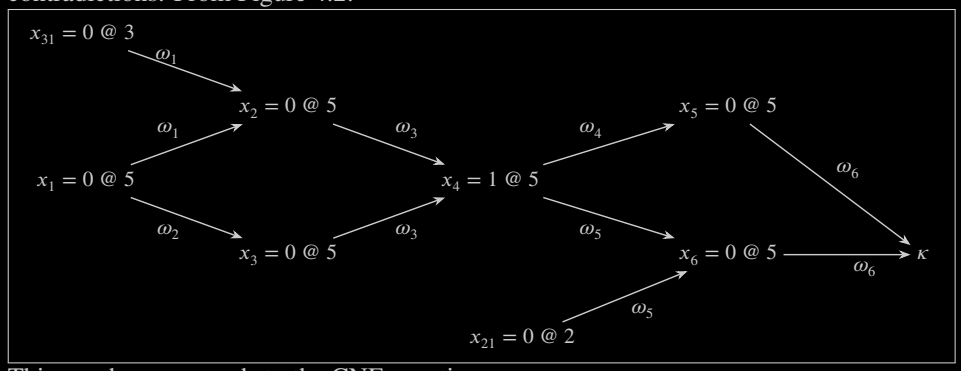
$$x_i = v@d$$

where i denotes the index of the variable being assigned, $v \in \{0, 1\}$ (alternatively, $v \in \{F, T\}$), and d is the decision level for which that assignment was executed.

So in our graph, $x_{31} = 0@3$ means variable 31 was assigned 0 (or False) at decision level 3. What about the edges? The edges are clauses, and they connect variables to each other, but what's the relationship? Let's go left to right in our implication graph. Variables 31 and 1 are assigned False at decision levels 3 and 5 respectively, and through clause 1, they both have an edge to variable 2, which is also assigned False at decision level 5. If we look at the original CNF equation, we see that in clause 1, x_1 and x_{31} are not negated, so they are False literals. That leaves x_2 Unassigned, so we propagate in order to satisfy the clause. We assign $x_2 = F$ because it's negated, so its literal will evaluate to True. Likewise, x_1 has another edge of clause 2 (ω_2) to x_3. If we look at

clause 2, we see that $x_1 = F$ forces $\neg x_3 = T \equiv x_3 = F$. Lastly, we see that x_2 and x_3 have an edge to x_4 through ω_3, forcing its assignment to be $x_4 = T$.

That was pretty easy to understand, so now let's look at an implication graph with contradictions. From Figure 4.2:



This graph corresponds to the CNF equation:

$$(\omega_1) \wedge (\omega_2) \wedge (\omega_3) \wedge (\omega_4) \wedge (\omega_5) \wedge (\omega_6) = (x_1 \vee x_{31} \vee \neg x_2)_1 \wedge (x_1 \vee \neg x_3)_2 \wedge (x_2 \vee x_3 \vee x_4)_3$$

$$\wedge (\neg x_4 \vee \neg x_5)_4 \wedge (x_{21} \vee \neg x_4 \vee \neg x_6)_5 \wedge (x_5 \vee x_6)_6$$

Up until $x_4 = 1@5$, this graph is the same as the one from above. After that, we have some more decisions and propagations, that I bet you can read by now, up until we reach this symbol: κ, which has two edges via ω_6. This node, κ, corresponds to a contradiction within clause ω_6 because its literals are all False.
When we resolve this conflict, we first start with the clause

$$(x_5 \vee x_6)$$

as that is the clause in which our conflict occurred. Next, we find the most recently propagated literal, which we will assume to be x_6 (the most recently propagated literal depends on our implementation of propagation, so for simplicity's sake, we will assume numerical order to settle ties in implication orderings on the same level). x_6 was propagated because of the clause

$$(x_{21} \vee \neg x_4 \vee \neg x_6)$$

Combining this with the current conflict clause yields:

$$(x_5 \vee x_6 \vee x_{21} \vee \neg x_4 \vee \neg x_6)$$

which we process to remove all instances of the conflicting variable, leading to a resolution clause of:

$$(x_5 \vee x_{21} \vee \neg x_4)$$

In this clause, the most recently propagated literal is x_5, so we grab its reason clause (ω_4) and combine once again to get:

$$(\neg x_4 \vee \neg x_5 \vee x_5 \vee x_{21} \vee \neg x_4)$$

Simplifying that clause gives us:

$$(\neg x_4 \vee x_{21})$$

The only propagated literal in the clause is x_4, so we grab its reason clause (ω_3) and combine again into:

$$(x_2 \vee x_3 \vee x_4 \vee \neg x_4 \vee x_{21})$$

This simplifies down into:

$$(x_2 \vee x_3 \vee x_{21})$$

Our most recently propagated literal is now x_3. Repeating the process again, we get an unsimplified resolution clause of:

$$(x_2 \vee x_3 \vee x_{21} \vee x_1 \vee \neg x_3)$$

Simplifying it down, we now get:

$$(x_2 \vee x_{21} \vee x_1)$$

Our last propagated literal in the clause is x_2. We combine our incomplete resolution clause with ω_1 to get:

$$(x_2 \vee x_{21} \vee x_1 \vee x_1 \vee x_{31} \vee \neg x_2)$$

This transforms into

$$(x_1 \vee x_{21} \vee x_{31})$$

which contains only decision literals.

What does this new clause, which we'll call ω_7, tell us? It tells us that all three decision literals **cannot** all be False (ie, at least one of them **must** be True).

4.1.2 Clause Learning - Results

Here are the results for Clause Learning:
uf20.91: **425ms**, up 98ms from Iterative DPLL
UUF50.218.1000: **9652ms**, up 3287ms from Iterative DPLL

Why did our runtime go up? Well, clause learning isn't easy, and the more contradictions we encounter, the more expensive clause learning we're performing. Another reason is that we're increasing the memory load every time we add in a new clause. Luckily, our testing sets are small, but on large problems, this would be a much larger issue. So then why do modern solvers use clause learning? Because there are ways to make it much faster. The first step would be to implement non-chronological backjumping, where instead of moving up just one decision level in our trail, we move up multiple at a time. The second step would be to implement First UIPs, which makes clause learning a less expensive operation (doesn't fix the memory load though), and it combined with non chronological backjumping allows us to instantly and fully use our learned clause to its potential. Although we have taken a small step back, we will see what leaps forward we will make in the next sections.

Aside: Why did we get rid of pure literal elimination? The problem with pure literal elimination is that once we start learning clauses, previously pure literals may no

longer be pure anymore once we add in a new clause. Because pure literals aren't born from decisions/ assignments, they aren't as "concrete" in their status. (Ie, propagation ***forces*** *new literal assignments, pure literal elimination is more of a "rule of thumb"). If we had a pure literal, say* 5*, and we added in a clause containing* -5*, unfortunately,* 5 *is no longer pure, and anything propagated because of* 5 *also would need to be undone because the reason why they were propagated/assigned those values in the first place doesn't hold anymore. Because we saw that pure literal elimination wasn't that helpful to begin with and with all these reasons as to why it wouldn't easily work under CDCL, it would just be much better to not have it at all.*

```
On all 1000 equations in uf20-91:

Clause Learning:           425ms (~0.43 seconds).
Iterative DPLL (AbP):      328ms (~0.33 seconds).
Iterative DPLL (PbA):      324ms (~0.32 seconds).
Watched Literals:          349ms (~0.35 seconds).
Pure Literal Elimination:  1548ms (~1.55 seconds).
Unit Propagation:          868ms (~0.87 seconds).
Recursive Backtracking:    1,540,613ms (~25.68 minutes).
Brute Force:               1,770,851ms (~29.51 minutes).
Short-Circuited (Basic):   49,874ms (~50 seconds).
Parallel:                  10,589ms (~10.5 seconds).

On all 1000 equations in UUF50.218.1000:

Clause Learning:          9,652ms (~9.65 seconds).
Iterative DPLL (AbP):     6,369ms (~6.37 seconds).
Iterative DPLL (PbA):     6,365ms (~6.37 seconds).
Watched Literals:         7,130ms (~7.13 seconds).
Pure Literal Elimination: 231,107ms (~3.85 minutes).
Unit Propagation:         140,041ms (~2.34 minutes).
```

4.2 Non-Chronological Backjumping

An issue I mentioned earlier with our initial clause learning algorithm is that we backtrack only one level per contradiction. Not one level at a time, just one level. The reason why this is an issue is that we generally never get to use what we learned from conflict resolution to flip certain assignments, which is why we still relied on the DPLL notion of "trying the other branch." Non-chronological backtracking, or "backjumping" is the new technique we'll use to fully exploit the learned clause.

We only need to make a very minimal interface change. Now that we won't use a "triedFalse" variable for our trail starts, our trail starts array can now just be an integer array where each element is just the index in the trail where a new decision level starts.

```
1       public interface:
2           SATSolver(string input)
3           bool solveCNF()
4
5       private interface:
6           ...
7           int[] trailStarts
8           ...
```

Next, since we're only changing the backtracking logic, we can keep everything else the same in our algorithm. Before we move onto code, answer this: what decision level should be backjump to after we learn a clause? Suppose we learn this clause:

$$(1 \vee 2 \vee \neg 4)$$

And each literal's decision level is its index (1 was assigned on decision level 1, 2 was assigned on decision level 2, 4 was assigned on level 4). What decision level should be backjump to? Should we backjump to 1? 2? Stay on 4? We want to backjump to decision level 2 because that way, every literal in the clause other than 4 will be False, and 4 will be Unassigned. That way, we can propagate immediately on the learned clause! So the rule for backjumping after learning a new clause is to backjump to the second largest decision level present in the clause. Here's how we'll do it:

Algorithm 32 CDCL with Backjumping

```
1:  procedure SOLVE
2:      setupFailed ← SETUP
3:      if setupFailed = true then return false
4:      end if
5:      decisionLevel ← 0
6:      while true do
7:          if ASSIGN(assignment) = true then
8:              return true
9:          else
10:             decisionLevel ← decisionLevel + 1
11:         end if
12:         succeeded ← PROPAGATE(assignment)
13:         if succeeded is not null then
14:             conflict ← value of succeeded
15:             learned ← RESOLVECONFLICT(conflict)
16:             if learned is empty then
17:                 return false
18:             end if
19:             if SIZE(trailStarts) = 0 then
20:                 return false
21:             end if
22:             maxLevel ← decisionLevel
23:             sLevel ← -1
24:             for each var in learned do
25:                 idx ← ABS(var)
26:                 dLvl ← trail[varToTrail[idx]].decisionLevel
27:                 if dLvl ≠ maxLevel then
28:                     sLevel ← MAX(sLevel, dLvl)
29:                 end if
30:             end for
31:             while trail is not empty ∧ trail.last.decisionLevel > sLevel do
32:                 remove entries from the trail
33:                 remove decision levels from trailStarts as necessary
34:             end while
35:             decisionLevel ← sLevel
36:             trailHead ← trailStarts.last
37:             CREATEWATCHED(learned, decisionLevel)
38:             continue
39:         end if
40:     end while
41: end procedure
```

The code remains largely the same. However, we now want to check that the learned clause isn't empty so we can avoid doing large amounts of work. When can the learned

clause become empty? To answer that question, let's answer this other question: "what does an empty clause mean?" From our section on Watched Literals, an empty clause means that the equation is unsatisfiable because no variable assignment can satisfy that clause. So if we learn an empty clause, that means that we're now learning that there is no satisfactory assignment of variables, thus our equation is unsatisfiable.

4.2.1 Non-Chronological Backjumping - Results

Okay so I know I said that we would make leaps forward in the next sections. I may have lied. Here are the results for Non-Chronological Backjumping:
uf20.91: **380ms**, down 45ms from Clause Learning.
UUF50.218.1000: **11291ms**, up 1639ms from Clause Learning.

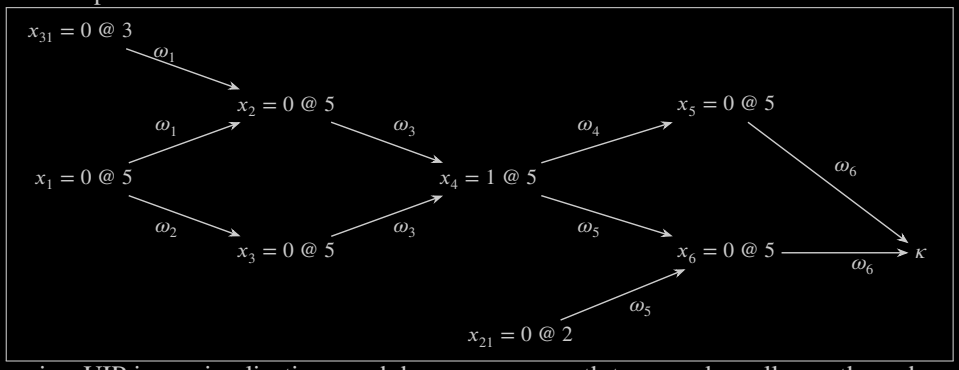
Why did our time to solve unsatisfiable equations go up so significantly? It has to do with the fact of how we're learning. We're not trying every assignment like brute force, but we are slowly learning that no matter what the assignment is, our equation is unsatisfiable. This takes a lot of backtracking and memory to learn, possibly attributing to our time increase. It's hard to say for certain.

```
On all 1000 equations in uf20-91:

Backjumping:               380ms (0.38 seconds).
Clause Learning:           425ms (~0.43 seconds).
Iterative DPLL (AbP):      328ms (~0.33 seconds).
Iterative DPLL (PbA):      324ms (~0.32 seconds).
Watched Literals:          349ms (~0.35 seconds).
Pure Literal Elimination:  1548ms (~1.55 seconds).
Unit Propagation:          868ms (~0.87 seconds).
Recursive Backtracking:    1,540,613ms (~25.68 minutes).
Brute Force:               1,770,851ms (~29.51 minutes).
Short-Circuited (Basic):   49,874ms (~50 seconds).
Parallel:                  10,589ms (~10.5 seconds).

On all 1000 equations in UUF50.218.1000:

Backjumping:              11,291ms (~11.29 seconds).
Clause Learning:          9,652ms (~9.65 seconds).
Iterative DPLL (AbP):     6,369ms (~6.37 seconds).
Iterative DPLL (PbA):     6,365ms (~6.37 seconds).
Watched Literals:         7,130ms (~7.13 seconds).
Pure Literal Elimination: 231,107ms (~3.85 minutes).
Unit Propagation:         140,041ms (~2.34 minutes).
```

4.3 First UIPs

Our next addition to CDCL is First UIPs. UIP stands for "unit implication point," which is called a “dominator” in our implication graph. What is a dominator? A node d is said to *dominate* another node n if every path in the graph from the entry node to n passes through d. Think of a binary tree. Our entry node is the the head of the tree, and it dominates its two children. Likewise, each of those children dominates their own children nodes. So in other words, the UIP in our implication graph is kind of like the reason why we had a contradiction in the first place!

Given that summary of what a UIP is, wouldn't the first UIP be the decision we made on the current decision level? After all, we wouldn't have had a contradiction if we didn't make that decision. Well not necessarily, because in our implication graph, every decision we make only has outward edges, whereas all propagated nodes have inward edges as well. So while it is true that a first UIP could be a decision variable, it's not true that it's *always* a decision variable. Take this previous implication graph as an example:



x_4 is a UIP in our implication graph because every path to x_5 and x_6 all pass through x_4's node.

So what does the first UIP offer us? Resolving clauses up until we encounter the first UIP maximizes the level we backjump to and it makes the clauses we learn shorter, not just in length, but also in how long it takes to resolve a clause. More details can be found here:

`https://www3.cs.stonybrook.edu/~cram/cse505/Fall20/Resources/cdcl.pdf`

How do we find the first UIP? The first UIP in a conflict is found when a the number of literals in the clause for the *current* decision level is 1. So if our current decision level is 6, we resolve only on literals from decision level 6 and stop once the clause contains only 1 literal from decision level 6.

4.3.1 Implementing First UIPs

Once again, we only need to touch the conflict resolution code:

Algorithm 33 Naive First UIP Conflict Resolution

```
1:  procedure RESOLVECONFLICT(conflict[1..n])
2:      curLevel ← decisionLevel
3:      end ← trail size
4:      begin ← 0
5:      resolveOn ← 0
6:      for i ← 1 up to n do
7:          var ← conflict[i]
8:          idx ← ABS(var)
9:          tIdx ← varToTrail[idx]
10:         if trail[tIdx].decisionLevel = curLevel then
11:             end ← MIN(end, tIdx)
12:             begin ← MAX(begin, tIdx)
13:             if trail[tIdx].reasonIdx exists then
14:                 resolveOn ← trail[tIdx].lit
15:             end if
16:         end if
17:     end for
18:     while begin ≠ end do
19:         lit ← ABS(resolveOn)
20:         tIdx ← varToTrail[resolveOn]
21:         reason ← Clauses[trail[tIdx].reasonIdx]
22:         APPEND(conflict, reason)
23:         remove all instances of lit and -lit from conflict
24:         remove all duplicate literals from conflict
25:         for each var in conflict do
26:             tIdx ← varToTrail[ABS(var)]
27:             if trail[tIdx].decisionLevel = curLevel then
28:                 end ← MIN(end, tIdx)
29:                 begin ← MAX(begin, tIdx)
30:                 if trail[tIdx].reasonIdx exists then
31:                     resolveOn ← trail[tIdx].lit
32:                 end if
33:             end if
34:         end for
35:     end while
36: end procedure
```

This is not a very efficient way of computing the first UIP (which you'll see in the results). However, let's analyze it anyways. It's the normal resolution code, but we do something different, which is stop when there's only one literal from the current decision level left in the clause. In our pseudocode, we don't compute the counts of literals from the current decision level, we have the condition $begin! = end$. Why is that equivalent? Well *end* is the index of the least recently propagated literal in the resolution clause, and *begin* is the index of the most recently propagated literal in the

resolution clause. So when the most recently and least recently propagated literals are the same, there must be only one left, therefore we can stop. Although it's clever, it's not efficient, and therefore, not very clever.

4.3.2 First UIPs - Results

Unfortunately our runtime increases again. At this point, I'm willing to accept that perhaps I was just really really good at implementing DPLL, but since I'm not good at implementing CDCL (I literally failed the first time), my implementation is not up to par. In the next section, I will identify possible improvements that can be made, and hopefully they'll make a *positive* difference.
uf20.91: **406ms**
UUF50.218.1000: **14012ms**

```
On all 1000 equations in uf20-91:

First UIPs:                406ms (~0.41 seconds)
Backjumping:               380ms (~0.38 seconds).
Clause Learning:           425ms (~0.43 seconds).
Iterative DPLL (AbP):      328ms (~0.33 seconds).
Iterative DPLL (PbA):      324ms (~0.32 seconds).
Watched Literals:          349ms (~0.35 seconds).
Pure Literal Elimination:  1548ms (~1.55 seconds).
Unit Propagation:          868ms (~0.87 seconds).
Recursive Backtracking:    1,540,613ms (~25.68 minutes).
Brute Force:               1,770,851ms (~29.51 minutes).
Short-Circuited (Basic):   49,874ms (~50 seconds).
Parallel:                  10,589ms (~10.5 seconds).

On all 1000 equations in UUF50.218.1000:

First UIPs:               14,012ms (~14.01 seconds).
Backjumping:              11,291ms (~11.29 seconds).
Clause Learning:          9,652ms (~9.65 seconds).
Iterative DPLL (AbP):     6,369ms (~6.37 seconds).
Iterative DPLL (PbA):     6,365ms (~6.37 seconds).
Watched Literals:         7,130ms (~7.13 seconds).
Pure Literal Elimination: 231,107ms (~3.85 minutes).
Unit Propagation:         140,041ms (~2.34 minutes).
```

4.4 A Second Round of Improvements

Ever since we started CDCL, I have been promising massive speedups over DPLL and yet our SAT solver has gotten slower each iteration. Before you treat me like Jesus and crucify me, [3] I knew and deep down you knew that the way we were implementing was not optimal to begin with. So in a small way, this is your fault too.

4.4.1 Removing Hashing

The first thing we're going to do is remove the maps and instead use arrays instead so we don't have to deal with hashing. Our interface becomes like so:

```
1    public interface:
2        SATSolver(string input)
3        bool solveCNF()
4
5    private interface:
6        ...
7        int[][] var_to_clause
8        (int, int)[] clause_to_var
```

In case you need a reminder as it has been a while, these are the data structures used to make watched literals happen. Before they were maps where the keys were variables or clause indices, but now, each index in the arrays are the keys and the values at those indices are, well, the values. There is a small implementation detail concerning *clause_to_var*, which is that now that the number of clauses are not fixed, everytime we add in a new clause, we need to add in a new entry to *clause_to_var* before we let our *createWatched(int, int)* overload handle assigning the values to that entry. Basically, to avoid going out of bounds for the data structure, we need to change the bounds before we access the new boundary. While we could let *createWatched(int, int)* handle the allocation as well, it would make it less versatile as it would have to only operate on the last clause in the database instead of any clause in the database.

The second point to mention is that now our propagation logic doesn't work because we need to create a mapping from literals (positive and negative) to indices (nonnegative). We use the following mapping:

$$2 * abs(lit) + (lit < 0)$$

This is how the mapping works out (left column is literal value, right column is index):

```
 1 → 0
-1 → 1
 2 → 2
-2 → 3
 3 → 4
-3 → 5
  ...
```

[3] My friends told me that it was too funny not to include so it stays.

The last step is to remove the duplicates set we use in clause resolution. We're going to add in an array of booleans to our SAT solver where index i is whether or not we've seen that variable during our clause resolution. However, since our (my) implementation of first UIP clause resolution is such a big fixer-upper, it deserves its own subsection.

4.4.2 Fixing Clause Resolution

Our current implementation of clause resolution is pretty long and inefficient. This was written before we understood implication graphs fully. Now we do understand implication graphs, so let's exploit that structure.

The idea is this: instead of rescanning over and over to make sure our resolution clause has just one current level variable, we'll keep track of which variables we've seen as well as which ones are from this decision level. Then, as we perform clause resolution, we'll update the seen and count variables to reflect the current state of the resolution clause, then once we've encountered our stopping condition, we'll use which variables we've kept marked as seen to construct our learned clause directly. This is very different from the way we've been performing clause resolution, which involved repeatedly scanning the resolution clause and recomputing how far along we are in the UIP process.

How are we using the implication graph in our new method? Well since the implication graph is an acyclic graph, that means we can use its structure for our UIP traversal. There is a relationship between the implication graph and our trail, in that our trail is just a compressed version of the implication graph, meaning it doesn't keep track of which variable assignments led to which propagations. However, they both follow the same structure in that the edge of our trail are the edges of our implication graphs and the order of variables in the trail and graphs are the same. Because the implication graph is acyclic, once we encounter a variable during clause resolution, we will never encounter it again while we're resolving. Thanks to that property, we can traverse the trail backwards without having to worry about going over the same parts of the trail/graph, which was something that we were sort of doing in the old method when we rescanned the whole conflict clause, whereas now, we are only concerned with the differences to the resolution clause made across each iteration.

Our resolution algorithm becomes as follows:

Algorithm 34 Improved First UIP

```
 1: procedure RESOLVECONFLICT(conflict[1..n])
 2:     levelCount ← 0
 3:     curLevel ← decisionLevel
 4:     for each var in conflict do
 5:         if seen[ABS(var)] = false then
 6:             seen[ABS(var)] ← true
 7:             tIdx ← varToTrail[ABS(var)]
 8:             if trail[tIdx].decisionLevel = curLevel then
 9:                 levelCount ← levelCount + 1
10:             end if
11:         end if
12:     end for
13:     for each entry in trail from last do          ▷ traverse the trail in reverse order
14:         idx ← entry.lit
15:         if seen[idx] = true then
16:             reason ← Clauses[entry.reasonIdx]
17:             for each var in reason do
18:                 if seen[ABS(var)] = false then
19:                     seen[ABS(var)] ← true
20:                     tIdx ← varToTrail[ABS(var)]
21:                     if trail[tIdx].decisionLevel = curLevel then
22:                         levelCount ← levelCount + 1
23:                     end if
24:                 end if
25:             end for
26:             seen[idx] ← false
27:             levelCount ← levelCount - 1
28:         end if
29:         if levelCount = 1 then
30:             break
31:         end if
32:     end for
33:     learned[1..m]
34:     for i ← 1 up to numVars do
35:         if seen[i] = true then
36:             lit ← trail[varToTrail[i]].lit
37:             APPEND(learned, −lit)
38:             seen[i] ← false
39:         end if
40:     end for
41:     return learned
42: end procedure
```

Here's the rundown of what's going on. In the first block, we're marking which variables

we've seen and how many of them are from the current decision level from the initial conflicting clause. Next, we start traversing our trail from the most recent trail entry, and we move towards the front of the trail. If we've seen the variable from the current trail entry, we grab its reason clause, and for each new unseen variable, we mark it as seen and increment the current decision level count. Next, we "remove" the variable we just processed by unmarking it as seen and decrementing *levelCount*.

Once we have only one literal from the current level, we construct a new learned clause by iterating over the entire *seen* array and we add the negative literal to the learned clause because the learned clause is supposed to learn the opposite of what we've done so we don't repeat it. Then we unmark everything we have seen so that way we can reuse the same array the next time there's a conflict instead of allocating a new one each time.

The results for this are very good, but I won't tell you what they are, because there's one more thing I want to do. You see, we iterate over the entire *seen* array, which is as large as the number of variables in our equation. However, it's very unlikely that we'll see many variables because first UIPs shrink how large the learned clause is, so there's less to be seen in the first place, but also because the more we learn, generally, the learned clause gets smaller because each new learned clause adds more constrictions on which assignments are valid, and as we constrict which ones are valid and which ones aren't, we inch towards learning the empty clause signaling that nothing is valid. This is just a long winded way to say that the iterating over the entire seen array each time, while O(n), could be better.

But isn't that microoptimizing? [4] After all, our two test sets have 20 and 50 variables respectively, which isn't much in the grand scheme of things. And you're right, our test sets are pretty small. However, the goal is to build a SAT solver that would scale well and most SAT solvers can handle problems with millions of variables pretty well, so I'd like this one to too. Making it scale well isn't the same as microoptimizing. Microoptimizing would be like saying $++i$ over $i++$. [5]

In order to do this, we're going to keep track of the indices of which variables we've seen in an array and then iterate over that array. We will also need to keep track of which indices are in which position so we can swap and pop for efficient removal of elements. [6] Our object now looks like this:

```
1       public interface:
2           SATSolver(string input)
3           bool solveCNF()
4
5       private interface:
6           ...
7           int[][] var_to_clause
8           (int, int)[] clause_to_var
```

[4] Microoptimizing is like cutting your hair to help you reach a weight loss goal, instead of fixing your running form so you'll be able to have better cardio.

[5] Pre vs post incrementation in loops generally does not affect runtime and whether or not it actually matters is heavily debated.

[6] I love swap and pop if only for the name. I encourage everyone to do swap and pops so we can say "swap and pop" more than we currently say "swap and pop."

```
 9
10          bool[] seen
11          int[] seenIdx
12          int[] seenPos
```

Here a subtle detail about these data structures. *seen* and *seenPos* are a fixed size of *numVars* because they require a full mapping from variable to value at all times, but *seenIdx* is dynamically sized because the number of variables that we see each clause resolution changes. So if you are implementing this yourself, make sure that *seen* and *seenPos* are allocated enough space upon initialization. Personally, I also reserve *numVars* amount of space in *seenIdx* so that it never has to reallocate space while solving, but it's not strictly necessary.

This is how these data structures change the algorithm:

Algorithm 35 Improved-improved First UIP

```
1:  procedure RESOLVECONFLICT(conflict[1..n])
2:      levelCount ← 0
3:      curLevel ← decisionLevel
4:      for each var in conflict do
5:          if seen[ABS(var)] = false then
6:              vIdx ← ABS(var)
7:              seen[vIdx] ← true
8:              APPEND(seenIdx, vIdx)
9:              seenPos[vIdx] ← SIZE(seenIdx)
10:             tIdx ← varToTrail[vIdx]
11:             if trail[tIdx].decisionLevel = curLevel then
12:                 levelCount ← levelCount + 1
13:             end if
14:         end if
15:     end for
16:     for each entry in trail from last do          ▷ traverse the trail in reverse order
17:         idx ← entry.lit
18:         if seen[idx] = true then
19:             reason ← Clauses[entry.reasonIdx]
20:             for each var in reason do
21:                 vIdx ← ABS(var)
22:                 if seen[ABS(var)] = false then
23:                     seen[vIdx] ← true
24:                     APPEND(seenIdx, vIdx)
25:                     seenPos[vIdx] ← SIZE(seenIdx)
26:                     tIdx ← varToTrail[vIdx]
27:                     if trail[tIdx].decisionLevel = curLevel then
28:                         levelCount ← levelCount + 1
29:                     end if
30:                 end if
31:             end for
32:             seen[idx] ← false
33:             pos ← seenPos[idx]
34:             last ← seenIdx.last
35:             seenIdx[pos] ← last
36:             seenIdx[last] ← pos
37:             REMOVELAST(seenIdx)
38:             levelCount ← levelCount - 1
39:         end if
40:         if levelCount = 1 then
41:             break
42:         end if
43:     end for
44:     learned[1..m]
45:     while seenIdx is not empty do
46:         lIdx ← seenIdx.last
47:         lit ← trail[varToTrail[lIdx]].lit
48:         APPEND(learned, −lit)
49:         seen[lIdx] ← false
50:         REMOVELAST(seenIdx)
51:     end while
52:     return learned
53: end procedure
```

In this code, not much changes other than that we index using the seenIdx and seenPos data structures to achieve a quicker deletion and modification of the learned clause by both swapping and popping seenIdx as well as seenPos.

4.4.3 Improvements - Results

I'll let the results speak for themselves: uf20.91: **264ms**
UUF50.218.1000: **2385ms**

```
On all 1000 equations in uf20-91:

Improved CDCL:             284ms (~0.28 seconds).
First UIPs:                406ms (~0.41 seconds).
Backjumping:               380ms (~0.38 seconds).
Clause Learning:           425ms (~0.43 seconds).
Iterative DPLL (AbP):      328ms (~0.33 seconds).
Iterative DPLL (PbA):      324ms (~0.32 seconds).
Watched Literals:          349ms (~0.35 seconds).
Pure Literal Elimination:  1548ms (~1.55 seconds).
Unit Propagation:          868ms (~0.87 seconds).
Recursive Backtracking:    1,540,613ms (~25.68 minutes).
Brute Force:               1,770,851ms (~29.51 minutes).
Short-Circuited (Basic):   49,874ms (~50 seconds).
Parallel:                  10,589ms (~10.5 seconds).

On all 1000 equations in UUF50.218.1000:

Improved CDCL:            2,385ms (~2.39 seconds).
First UIPs:               14,012ms (~14.01 seconds).
Backjumping:              11,291ms (~11.29 seconds).
Clause Learning:          9,652ms (~9.65 seconds).
Iterative DPLL (AbP):     6,369ms (~6.37 seconds).
Iterative DPLL (PbA):     6,365ms (~6.37 seconds).
Watched Literals:         7,130ms (~7.13 seconds).
Pure Literal Elimination: 231,107ms (~3.85 minutes).
Unit Propagation:         140,041ms (~2.34 minutes).
```

Chapter 5

VSIDS

Now that we've finished implementing CDCL, these next chapters will be focused on other algorithms and data structures used to improve the performance of CDCL. We will start with one of the easier methods of improving CDCL: VSIDS.

Variable State Independent Decaying Sum (VSIDS) is a type of variable assignment heuristic, basically a loosely defined rule, that is used in modern CDCL solvers. It is *the* best variable assignment/branching heuristic with many variations. In this chapter, we follow this paper

`https://arxiv.org/pdf/1506.08905`

which gives a much more in depth and theoretical explanation of how and why they work. Four our purposes, we'll understand at a higher level what VSIDS are as well as how and why they work.

Currently, our variable assignment choosing heuristic has been "find first Unassigned variable, and assign it True." With VSIDS, this becomes "find the most active Unassigned variable, and assign it True." The first step in VSIDS is to add in an array of **activity scores** which rank "how active" a variable is. There are two schemes for ranking activity score, which is ranking each literal (positive and negative variables), or just the variables (absolute value of the literal). For our purposes, we will rank just the variables.

The next step is after clause resolution, add to each variable's score if it was involved in clause resolution. Again, there are multiple ways to do this. Some implementations choose only the variables in the learned clause, some implementations choose every variable involved in the resolution algorithm, even if they are resolved away, and there are more implementations than just those two. For our purposes, we will choose bumping the scores of only the variables in the learned clause.

The last step, which doesn't happen every iteration, is to multiply each score by a decay factor [1] after a certain number of conflicts. The idea behind this is to even the playing field for each variable. If a variable accumulates a giant activity score, obviously it will be the first pick if it's valid. However, activity scores don't represent how recent

[1] Not to be confused with Deadpool's "dying factor."

the activity score bump occurred, so if that variable stopped being active, it would still be the first pick despite not actually being active relative to the learned clauses. This decay lets the variables that are starting to become more active have a better chance at competing with the potentially inactive, but score-dominating variables. As it is a decaying factor, this value is chosen to be greater than 0, but less than 1, so that the scores shrink to some percentage.

5.1 cVSIDS

We're going to start out with implementing the most basic form of VSIDS and work our way up to some better versions. We'll start with the "textbook" form, which was outlined in the section's introduction. This form of VSIDS is called **cVSIDS** by the paper we're using. We first add in our *activity* array into our solver object with a counter for the number of conflicts.

```
1       public interface:
2           SATSolver(string input)
3           bool solveCNF()
4
5       private interface:
6           ...
7           double[] activity
8           int numConflicts
9           ...
```

Then our solve code becomes as follows:

Algorithm 36 cVSIDS Solving Method

```
1:  procedure SOLVE
2:      SETUP
3:      while true do
4:          if trailHead = size of trail then
5:              maxScore ← -1.0
6:              firstUnassigned ← -1
7:              for i ← 1 up to numVars do
8:                  if assignment[i] = Unassigned then
9:                      if maxScore < activity[i] then
10:                         firstUnassigned ← i
11:                         maxScore ← activity[i]
12:                     end if
13:                 end if
14:             end for
15:             ASSIGN(firstUnassigned)
16:         end if
17:         propResult ← PROPAGATE(assignment)
18:         if propResult failed then
19:             numConflicts ← numConflicts + 1
20:             RESOLVECONFLICT(conflict)
21:             BACKJUMP(backJumpTo)
22:                                                  ▷ apply decay factor
23:             if numConflicts = MAX_CONFLICTS then
24:                 for i ← 1 up to numVars do
25:                     activity[i] ← activity[i] * DECAY
26:                 end for
27:                 numConflicts ← 0
28:             end if
29:             continue
30:         end if
31:     end while
32: end procedure
```

Algorithm 37 cVSIDS First UIP resolution

```
1:  procedure RESOLVECONFLICT(conflict[1..n])
2:      ...
3:      while seenIdx is not empty do
4:          lIdx ← seenIdx.last
5:          activity[lIdx] ← activity[lIdx] + 1
6:          ...
7:      end while
8:  end procedure
```

The pseudocode is very straightforward, so I won't bother to explain it much. Just so you know, if you choose to initialize *maxScore* with a value of 0.0 in the assignment code, use >= instead of > because the activity scores start out at 0.0, so not using >= will mess with unsatisfiable problems. In our code to apply the decay factor, I use "X" and "Y" to represent the number of conflicts and the decay factor respectively. The reason why these aren't set values, but rather placeholders is because there aren't two numbers that perform the best for *all* equations, so they should be adjusted programmatically or manually set experimentally. However, I just decided to choose two random numbers, so I used $\mathbf{X = 10}$ and $\mathbf{Y = 0.8}$. But results for different X and Y will be shown as well.

The results were as follows: uf20.91: **279ms**
UUF50.218.1000: **1841ms**

We didn't get a big speedup for satisfiable problems, but we did shave off about half a second for the unsatisfiable problems, which I think is pretty darn good.

```
On all 1000 equations in uf20-91:

cVSIDS (X = 1, Y = 0.5):    260ms (~0.26 seconds).
cVSIDS (X = 1, Y = 0.8):    266ms (~0.27 seconds).
cVSIDS (X = 10, Y = 0.5):   270ms (~0.27 seconds).
cVSIDS (X = 10, Y = 0.8):   279ms (~0.29 seconds).
Improved CDCL:              284ms (~0.28 seconds).

On all 1000 equations in UUF50.218.1000:

cVSIDS (X = 1, Y = 0.5):    1,680ms (~1.68 seconds).
cVSIDS (X = 1, Y = 0.8):    1,705ms (~1.71 seconds).
cVSIDS (X = 10, Y = 0.5):   1,782ms (~1.78 seconds).
cVSIDS (X = 10, Y = 0.8):   1,841ms (~1.84 seconds).
Improved CDCL:              2,385ms (~2.39 seconds).
```

In the results, since we will be interating over many versions of VSIDS, I will leave out the older results and only compare against the previous versions of our solver. In the results, it appears to be that lower X and Y values lead to faster solving times, however, I encourage you to choose your own X and Y values and see what happens for yourself.

5.2 mVSIDS

mVSIDS is the second form of VSIDS introduced by the paper from the section introduction. In this type of VSIDS, we bump the values of every variable we resolve on during clause resolution as well as the values of the variables in the learned clause. The multiplicative decay step is applied the same as during cVSIDS. This means that we only need to change the clause resolution code.

The algorithmic change is basically trivial:

Algorithm 38 mVSIDS First UIP resolution

```
1:  procedure RESOLVECONFLICT(conflict[1..n])
2:      ...
3:      for each entry in trail from last do          ▷ traverse the trail in reverse order
4:          idx ← entry.lit
5:          if seen[idx] = true then
6:              reason ← Clauses[entry.reasonIdx]
7:              for each var in reason do
8:                  ...
9:              end for
10:             ...
11:             activity[idx] ← activity[idx] + 1
12:         end if
13:         if levelCount = 1 then
14:             break
15:         end if
16:     end for
17:     while seenIdx is not empty do
18:         lIdx ← seenIdx.last
19:         activity[lIdx] ← activity[lIdx] + 1
20:         ...
21:     end while
22: end procedure
```

Here are the results using $\mathbf{X = 10}$ and $\mathbf{Y = 0.8}$:

```
On all 1000 equations in uf20-91:

mVSIDS (X = 1, Y = 0.5):    279ms (~0.28 seconds).
mVSIDS (X = 1, Y = 0.8):    277ms (~0.28 seconds).
mVSIDS (X = 10, Y = 0.5):   269ms (~0.27 seconds).
mVSIDS (X = 10, Y = 0.8):   284ms (~0.28 seconds).

cVSIDS (X = 1, Y = 0.5):    260ms (~0.26 seconds).
cVSIDS (X = 1, Y = 0.8):    266ms (~0.27 seconds).
cVSIDS (X = 10, Y = 0.5):   270ms (~0.27 seconds).
cVSIDS (X = 10, Y = 0.8):   279ms (~0.29 seconds).

Improved CDCL:              284ms (~0.28 seconds).

On all 1000 equations in UUF50.218.1000:

mVSIDS (X = 1, Y = 0.5):    1,532ms (~1.53 seconds).
mVSIDS (X = 1, Y = 0.8):    1,526ms (~1.53 seconds).
mVSIDS (X = 10, Y = 0.5):   1,420ms (~1.42 seconds).
mVSIDS (X = 10, Y = 0.8):   1,467ms (~1.47 seconds).

cVSIDS (X = 1, Y = 0.5):    1,680ms (~1.68 seconds).
cVSIDS (X = 1, Y = 0.8):    1,705ms (~1.71 seconds).
cVSIDS (X = 10, Y = 0.5):   1,782ms (~1.78 seconds).
cVSIDS (X = 10, Y = 0.8):   1,841ms (~1.84 seconds).

Improved CDCL:              2,385ms (~2.39 seconds).
```

So even with the worst mVSIDS time, it beats the best cVSIDS time (per the benchmark, results may vary, these tests are very small and are not very representative of real-world SAT solver usage).

5.3 EMA/Normalized VSIDS

EMA stands for "Exponential Moving Average" and it's what allows us to mathematically decide which variables' activity scores should be decayed and by how much. If we go back to the earlier example of some variable gaining a really high activity score and then becoming inactive, but still dominating over the more recently active variables, this is what EMA helps with. EMA will decay high scoring, but actually inactive variables' scores while mostly preserving the actually active variables' scores.

The way it does it is with this nifty formula:

$$s_n = (1 - f)\delta_n + f * s_{n-1}$$

s_n is the **current score**, f is our **decay factor**, $\delta_n \in \{0, 1\}$ where a value of 0 corresponds to a variable not being bumped, and a value of 1 corresponds to a variable being bumped. Let's walk through an example:

Suppose $f = 0.95$, and $s_0 = 0.0$. In the first conflict ($n = 1$), a variable was involved. Its score becomes:

$$s_1 = (1 - 0.95)(1) + 0.95(0.0) = 0.05$$

It's not much, but it's much more than 0, so this variable is considered high ranking. Now suppose in the next conflict ($n = 2$), the variable was not involved. Its score now becomes:

$$s_2 = (1 - 0.95)(0) + 0.95(0.05) = 0.0475$$

This is less than 0.05, so now it's low ranking, even if the difference is small. Now suppose it's involved in the third conflict ($n = 3$):

$$s_3 = (1 - 0.95)(1) + 0.95(0.0475) = 0.095125$$

This is much larger than what it was before.

Although you probably have guessed already, we've been implementing VSIDS in a straightforward naive way by following the described process down to the letter. For sake of simplicity, for Normalized VSIDS, we'll also implement it naively.

Implementing normalized VSIDS isn't too different to how we're already implementing it, except now, we update every score upon each conflict iteration. We'll need an additional data structure to keep track of the δ values. Since we're appending normalization on top of mVSIDS, we still update a δ value only when we resolve a variable or it appears in the learned clause. This means that we now delay the updating of our activity scores until after we construct the learned clause:

Algorithm 39 cVSIDS Solving Method

```
1:  procedure SOLVE
2:      SETUP
3:      while true do
4:          if trailHead = size of trail then
5:              maxScore ← -1.0
6:              firstUnassigned ← -1
7:              for i ← 1 up to numVars do
8:                  if assignment[i] = Unassigned then
9:                      if maxScore < activity[i] then
10:                         firstUnassigned ← i
11:                         maxScore ← activity[i]
12:                     end if
13:                 end if
14:             end for
15:             ASSIGN(firstUnassigned)
16:         end if
17:         propResult ← PROPAGATE(assignment)
18:         if propResult failed then
19:             numConflicts ← numConflicts + 1
20:             RESOLVECONFLICT(conflict)
21:             BACKJUMP(backJumpTo)
22:                                                  ▷ apply decay factor
23:             for i ← 1 up to numVars do
24:                 if delta[i] = true then
25:                     activity[i] ← (1 - Y) + DECAY * activity[i]
26:                 else
27:                     activity[i] ← DECAY * activity[i]
28:                 end if
29:                 delta[i] ← false
30:             end for
31:             numConflicts ← 0
32:             continue
33:         end if
34:     end while
35: end procedure
```

Algorithm 40 Normalized VSIDS First UIP resolution

```
1:  procedure RESOLVECONFLICT(conflict[1..n])
2:      ...
3:      for each entry in trail from last do          ▷ traverse the trail in reverse order
4:          idx ← entry.lit
5:          if seen[idx] = true then
6:              reason ← Clauses[entry.reasonIdx]
7:              for each var in reason do
8:                  ...
9:              end for
10:             ...
11:             delta[idx] ← delta[idx]
12:             activity[idx] ← activity[idx] + 1
13:         end if
14:         if levelCount = 1 then
15:             break
16:         end if
17:     end for
18:     while seenIdx is not empty do
19:         lIdx ← seenIdx.last
20:         activity[lIdx] ← activity[lIdx] + 1
21:         delta[lIdx] ← true
22:         ...
23:     end while
24: end procedure
```

Notice how that there is no X value anymore because normalization happens upon every iteration instead of after a set number. Thankfully, our fastest times were when we updated activity scores after every iteration anyways, so we probably weren't missing some "fastest" time under a different X value.

Here are the results:

```
On all 1000 equations in uf20-91:

EMA (X = 1, Y = 0.95):      266ms (~0.27 seconds).
EMA (X = 1, Y = 0.8):       287ms (~0.29 seconds).
EMA (X = 1, Y = 0.5):       280ms (~0.28 seconds).

mVSIDS (X = 1, Y = 0.5):    279ms (~0.28 seconds).
mVSIDS (X = 1, Y = 0.8):    277ms (~0.28 seconds).
mVSIDS (X = 10, Y = 0.5):   269ms (~0.27 seconds).
mVSIDS (X = 10, Y = 0.8):   284ms (~0.28 seconds).

cVSIDS (X = 1, Y = 0.5):    260ms (~0.26 seconds).
cVSIDS (X = 1, Y = 0.8):    266ms (~0.27 seconds).
cVSIDS (X = 10, Y = 0.5):   270ms (~0.27 seconds).
cVSIDS (X = 10, Y = 0.8):   279ms (~0.29 seconds).

Improved CDCL:              284ms (~0.28 seconds).

On all 1000 equations in UUF50.218.1000:

EMA (X = 1, Y = 0.95):      1,504ms (~1.50 seconds).
EMA (X = 1, Y = 0.8):       1,497ms (~1.50 seconds).
EMA (X = 1, Y = 0.5):       1,621ms (~1.62 seconds).

mVSIDS (X = 1, Y = 0.5):    1,532ms (~1.53 seconds).
mVSIDS (X = 1, Y = 0.8):    1,526ms (~1.53 seconds).
mVSIDS (X = 10, Y = 0.5):   1,420ms (~1.42 seconds).
mVSIDS (X = 10, Y = 0.8):   1,467ms (~1.47 seconds).

cVSIDS (X = 1, Y = 0.5):    1,680ms (~1.68 seconds).
cVSIDS (X = 1, Y = 0.8):    1,705ms (~1.71 seconds).
cVSIDS (X = 10, Y = 0.5):   1,782ms (~1.78 seconds).
cVSIDS (X = 10, Y = 0.8):   1,841ms (~1.84 seconds).

Improved CDCL:              2,385ms (~2.39 seconds).
```

Here we see that EMA/normalizing the VSIDS doesn't make a huge contribution, but it still makes a decent one for some values of Y.

5.4 AMA/adaptVSIDS

Adaptive Moving Average (AMA) VSIDS is an extension to EMA VSIDS where rather than having a fixed decay factor, we choose between two different decay rates depending on the state of our learned clause. The big idea behind making the decay factor variable instead of fixed is that our solver's "productivity" isn't fixed. The learned clause affects

this productivity in how high the quality of the learned clause is.

To figure out the quality of a learned clause, we have to look at its **LBD** which stands for **literal blocks distance**. LBD is defined as the number of distinct decision levels in the learned clause. A learned clause with a low LBD is considered to be higher quality than that of one with a high LBD. The paper doesn't explicitly say why this is the case, [2] but we can make a few guesses as to why. Considering that a low LBD means that there aren't many decision levels involved in the contradiction, so that means that the contradiction is more or less "confined" to a smaller area of the search space. This indicates that the conflict has a more localized relationship with the current set of decisions. Such clauses have a higher chance to become unit or nearly unit, under a smaller amount of changes to the current assignment, better contraining future search more effectively. Contrast this to high LBD, which has multiple decision levels involved in the contradiction, meaning that the contradiction spans a broader portion of the current search space, making it less likely it'll have an immediate effect on constraining future search.

This is supported by the fact that when we learn a clause with a low LBD, we choose a higher decay factor, so that variable activity scores shrink less, meaning we remember them more in the present. Likewise, when we learn a clause with a high LBD, we choose a lower decay factor, so variable activity scores shrink faster, making older activity lose its influence faster relative to the newer active variables.

The paper introduces this variable called *lbdema*, which is defined as the exponential moving average of the LBD. Like our EMA VSIDS, this is calculated upon every iteration, and it's also a recurrence relation where the previous value affects what the next value will be. The formula is similar to the previous EMA formula:

$$lbdema_n = (1 - \alpha) * LBD_t + \alpha * lbdema_{n-1}$$

α is our **smoothing factor**, LBD_t is the LBD of the current learned clause, and $lbdema_{n-1}$ is the previous lbdema. In our testing of AMA VSIDS, I used $\alpha = 0.1$.

5.5 VSIDS - Results

The results for VSIDS vary wildly because there are so many different types beyond what we've explored, and there are so many different parameters that could affect the end results. For VSIDS, although there might look like there's a clear winner, we need to keep in mind that we're using two relatively small test sets, so drawing an actual conclusion as to which VSIDS version is "objectively" better isn't credible. Although AMA VSIDS isn't the fastest from our benchmarks, it will be the version we continue with for the rest of our paper. As for code, given how easy it is to swap out each version of VSIDS with each other, the exact code used for each version won't be provided, as the total changes were small enough that following along shouldn't be overly difficult.

Another thing to note is that at the moment, there was not much exploration for the time differences. In order to figure out why the speeds between each version was different, it would be helpful to analyze how many conflicts there were, as well as the order of deciding variables. As of now, I am just looking at the results and just accepting

[2] I might be illiterate and may have just missed the reason.

them as they are, not because I lack curiosity or scientific rigor, but because this is the second night in a row where I barely got any sleep for no reason, and I am too tired to analyze this under a microscope.

```
On all 1000 equations in uf20-91:

AMA (Y = 0.75 or 0.99):     274ms (~0.27 seconds).

EMA (X = 1, Y = 0.95):      266ms (~0.27 seconds).
EMA (X = 1, Y = 0.8):       287ms (~0.29 seconds).
EMA (X = 1, Y = 0.5):       280ms (~0.28 seconds).

mVSIDS (X = 1, Y = 0.5):    279ms (~0.28 seconds).
mVSIDS (X = 1, Y = 0.8):    277ms (~0.28 seconds).
mVSIDS (X = 10, Y = 0.5):   269ms (~0.27 seconds).
mVSIDS (X = 10, Y = 0.8):   284ms (~0.28 seconds).

cVSIDS (X = 1, Y = 0.5):    260ms (~0.26 seconds).
cVSIDS (X = 1, Y = 0.8):    266ms (~0.27 seconds).
cVSIDS (X = 10, Y = 0.5):   270ms (~0.27 seconds).
cVSIDS (X = 10, Y = 0.8):   279ms (~0.29 seconds).

Improved CDCL:              284ms (~0.28 seconds).

On all 1000 equations in UUF50.218.1000:

AMA (Y = 0.75 or 0.99):     1,629ms (~1.63 seconds).

EMA (X = 1, Y = 0.95):      1,504ms (~1.50 seconds).
EMA (X = 1, Y = 0.8):       1,497ms (~1.50 seconds).
EMA (X = 1, Y = 0.5):       1,621ms (~1.62 seconds).

mVSIDS (X = 1, Y = 0.5):    1,532ms (~1.53 seconds).
mVSIDS (X = 1, Y = 0.8):    1,526ms (~1.53 seconds).
mVSIDS (X = 10, Y = 0.5):   1,420ms (~1.42 seconds).
mVSIDS (X = 10, Y = 0.8):   1,467ms (~1.47 seconds).

cVSIDS (X = 1, Y = 0.5):    1,680ms (~1.68 seconds).
cVSIDS (X = 1, Y = 0.8):    1,705ms (~1.71 seconds).
cVSIDS (X = 10, Y = 0.5):   1,782ms (~1.78 seconds).
cVSIDS (X = 10, Y = 0.8):   1,841ms (~1.84 seconds).

Improved CDCL:              2,385ms (~2.39 seconds).
```